

Experiment Update

Mazdak Teymourian

March 6, 2025

Experiments

- New FGSM
- Testing New FGSM Robustness
- Mix Attack
- Curv FGSM
- Importance of Alpha Function
- FGSM Alpha Drop
- Scheduling FGSM Alpha During Training
- Effect of BatchNorm on CO

New FGSM

- l_2 Instead of l_∞
- Random start
- No epsilon ball clipping
- No max range clipping

New FGSM

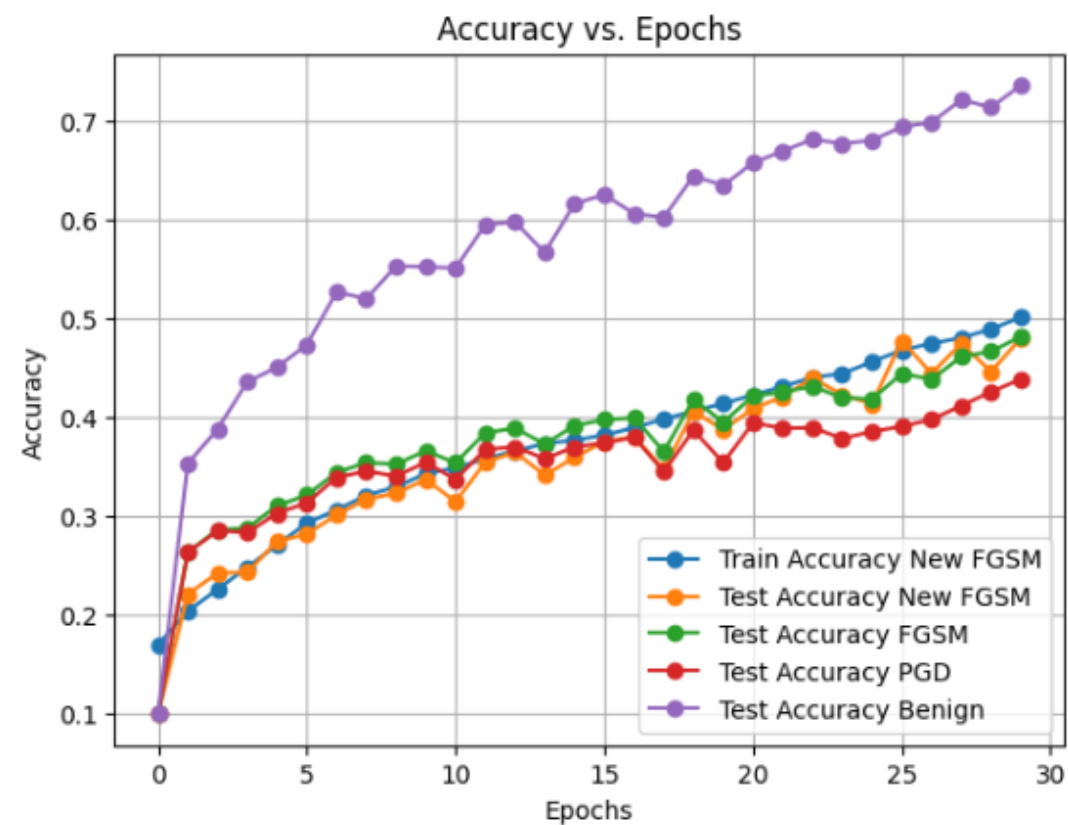
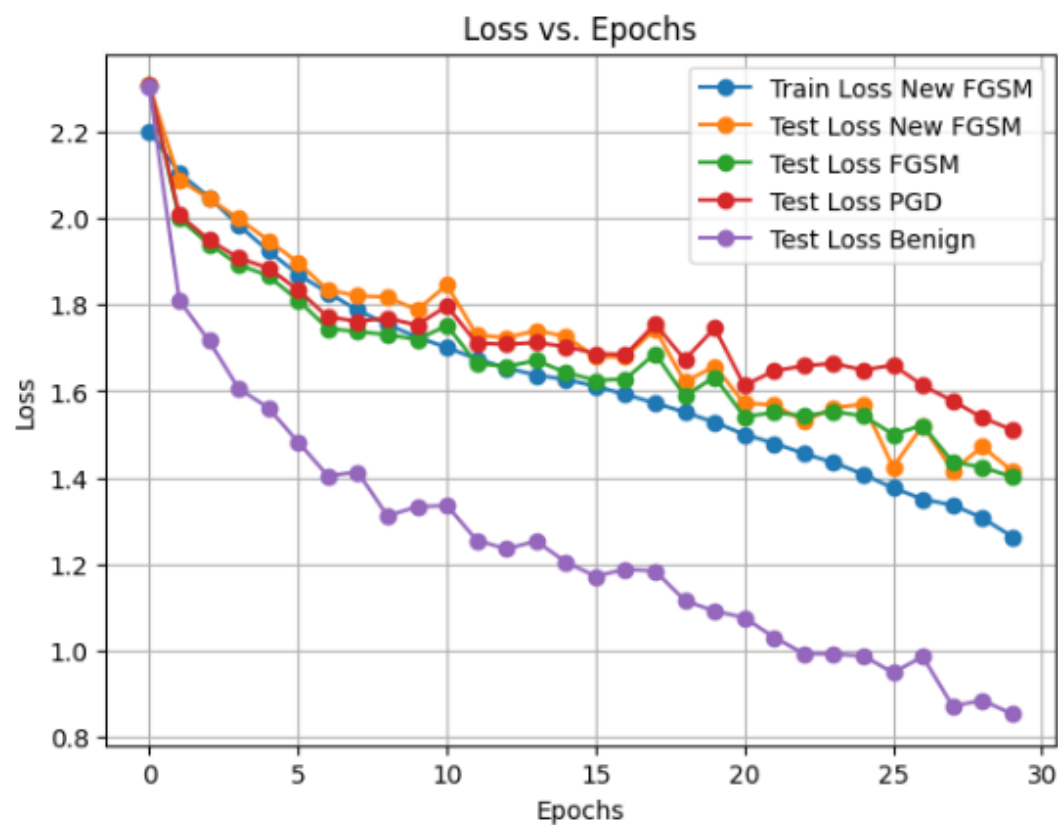
```
def new_fgsm_attack(x, y, model, epsilon, alpha, k = 2, noise=True, max_range_clamp=False):  
    epsilon = epsilon / std  
    alpha = alpha / std  
    delta = torch.zeros_like(x).cuda()  
    if k > 0:  
        for j in range(len(epsilon)):  
            delta[:, j, :, :].uniform_(-k * epsilon[j][0][0].item(), k * epsilon[j][0][0].item())  
    if max_range_clamp:  
        delta = clamp(delta, lower_limit - x, upper_limit - x)  
    delta.requires_grad = True  
    output = model(x + delta[:x.size(0)])  
    loss = F.cross_entropy(output, y)  
    loss.backward()  
    grad = delta.grad.detach()  
    scaled_grad = scale_grad(grad)  
    delta = delta + alpha * scaled_grad  
    if max_range_clamp:  
        delta = clamp(delta, lower_limit - x, upper_limit - x)  
    return x + delta
```

New FGSM

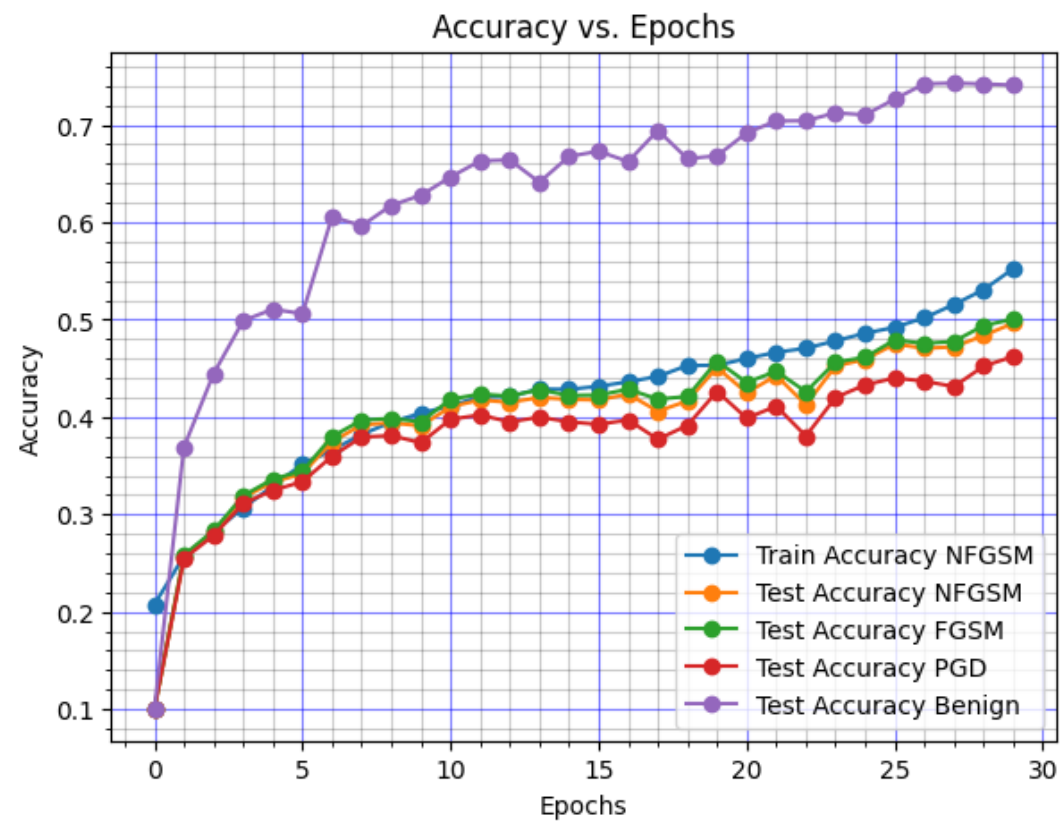
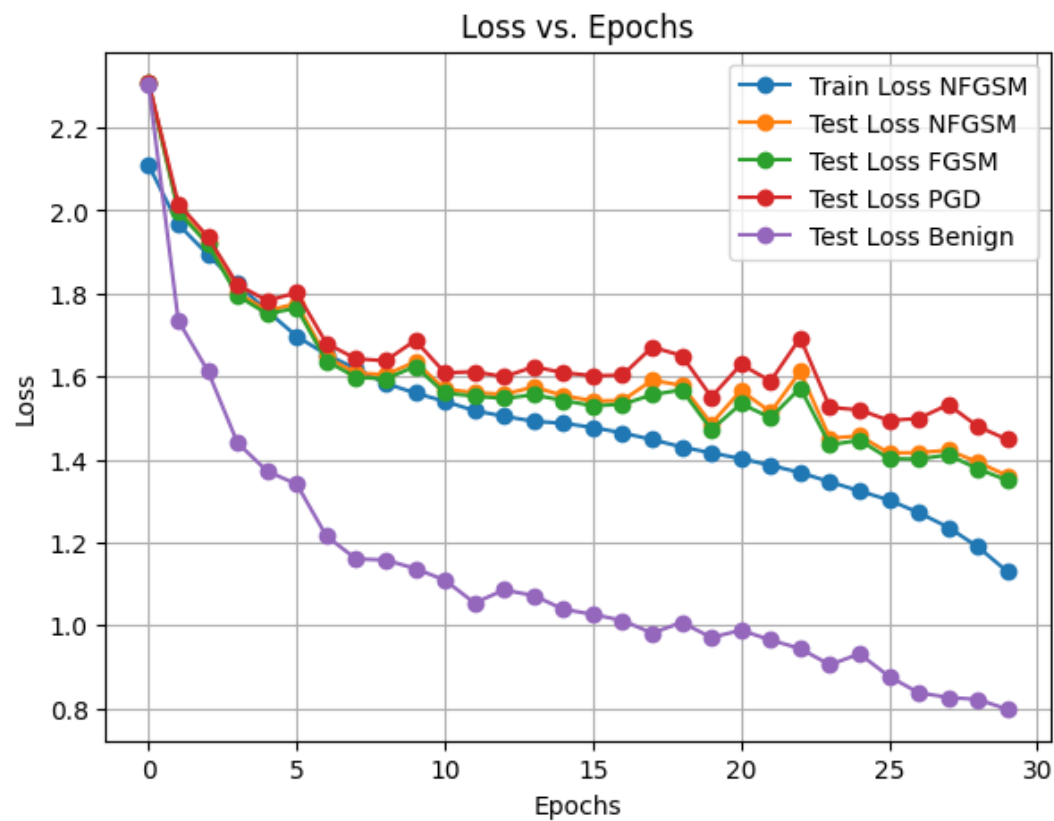
- Scaling gradient values between -1 and 1

```
#Scales the grad values between -1 and 1 without changing the sign  
#Doesn't clip the delta  
def scale_grad(grad):  
    flattened_g = grad.view(grad.shape[0], grad.shape[1], -1, 1)  
    ming, _ = torch.min(flattened_g, dim=2, keepdim=True)  
    maxg, _ = torch.max(flattened_g, dim=2, keepdim=True)  
    absming = torch.abs(ming)  
    absmaxg = torch.maximum(absming, maxg)  
    return grad / absmaxg
```

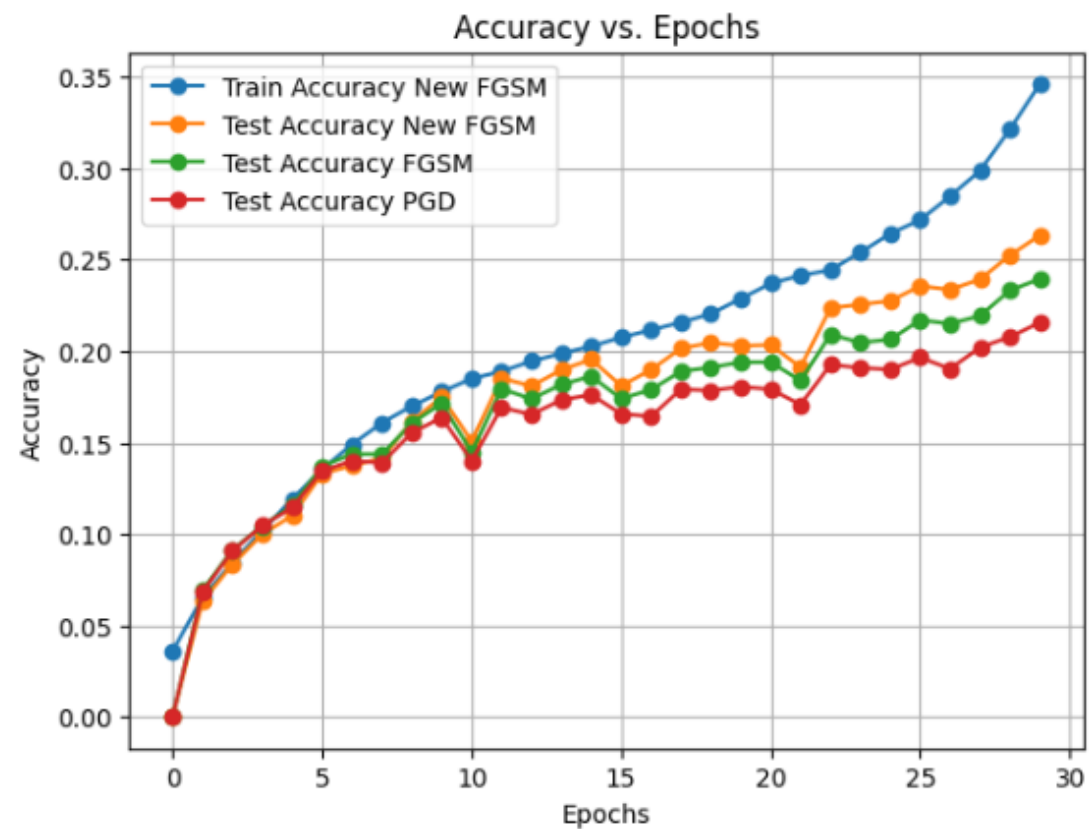
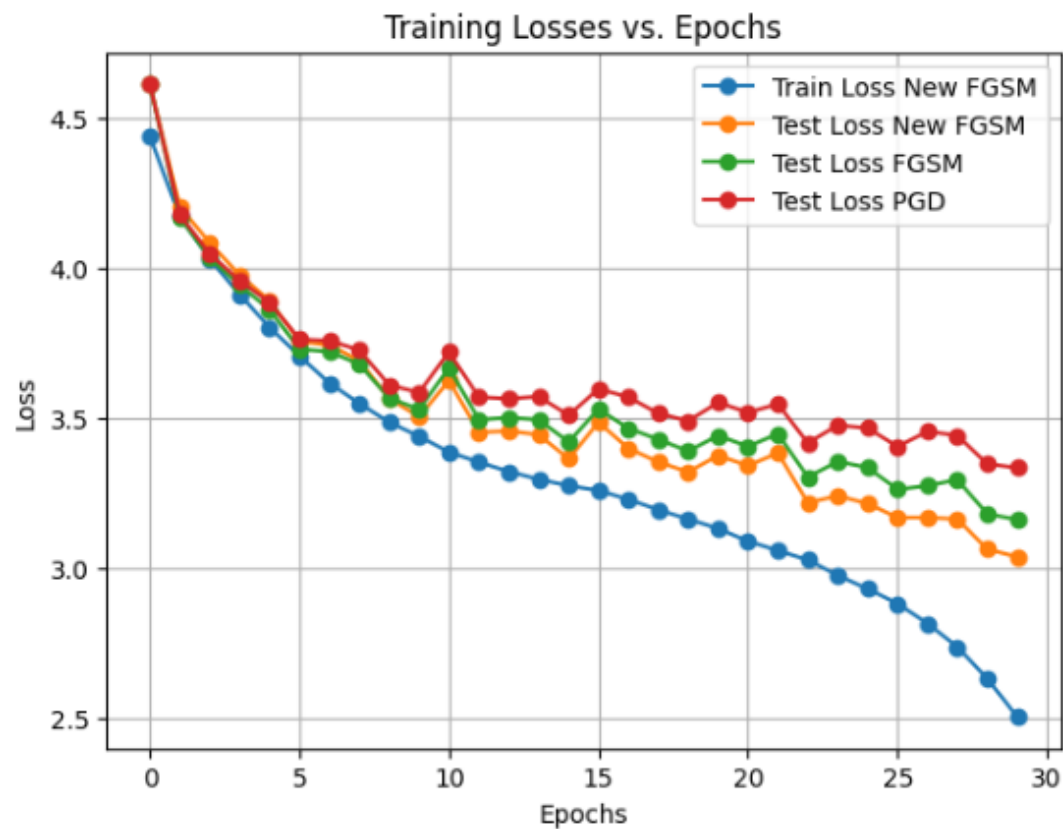
New FGSM Performance



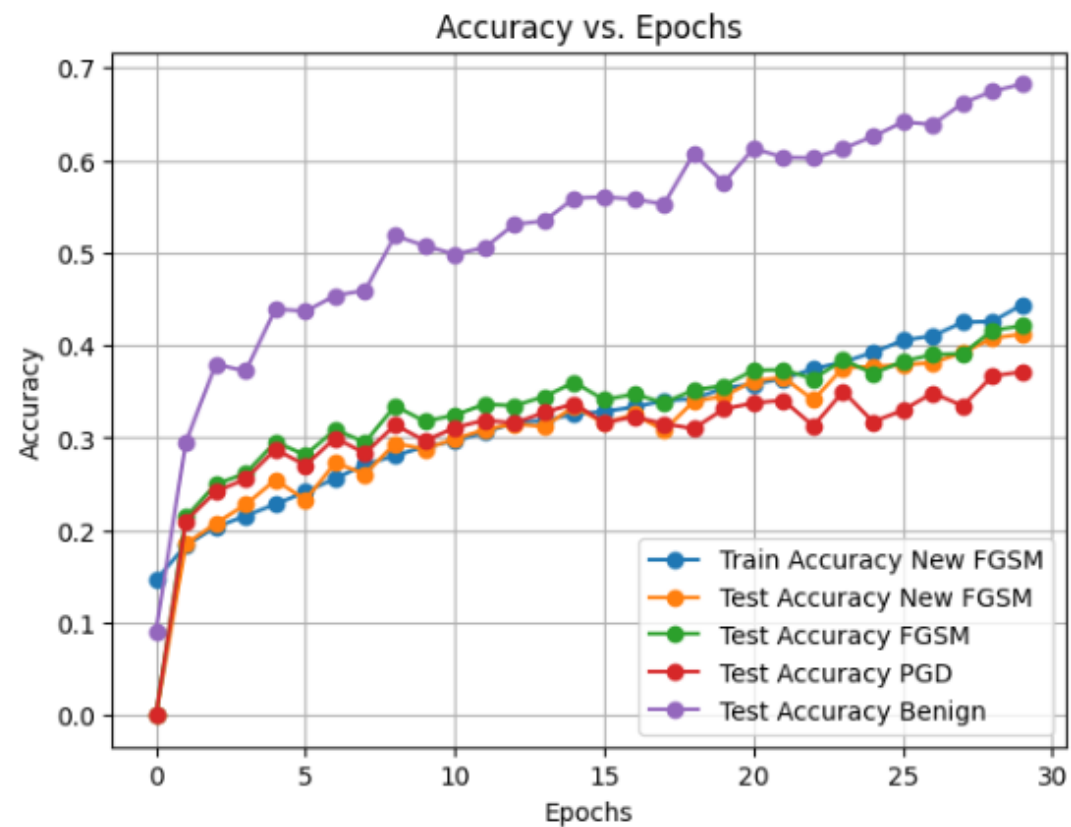
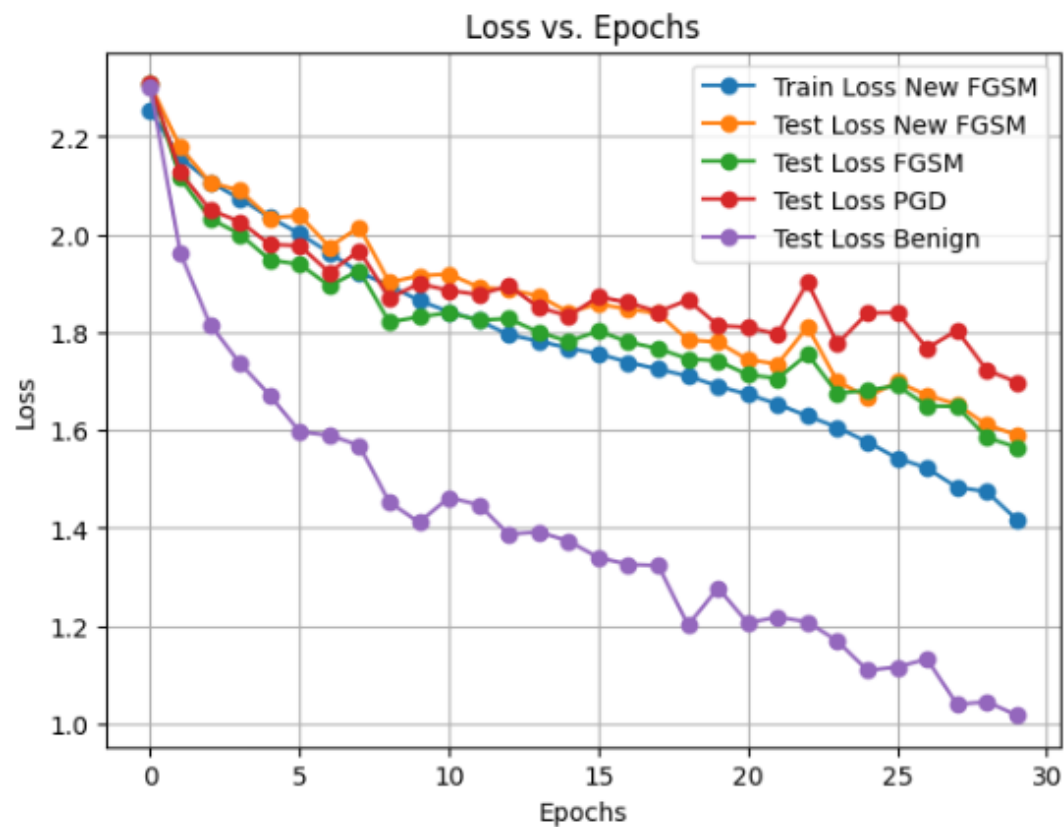
NFGSM Performance



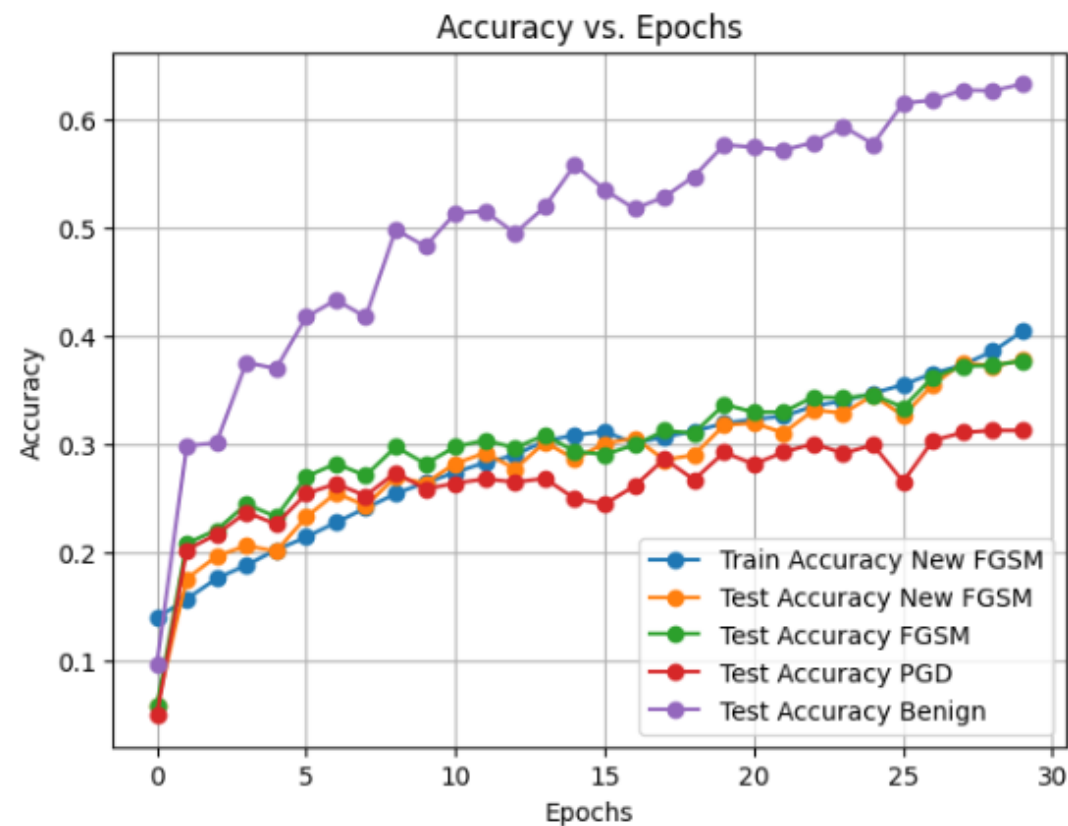
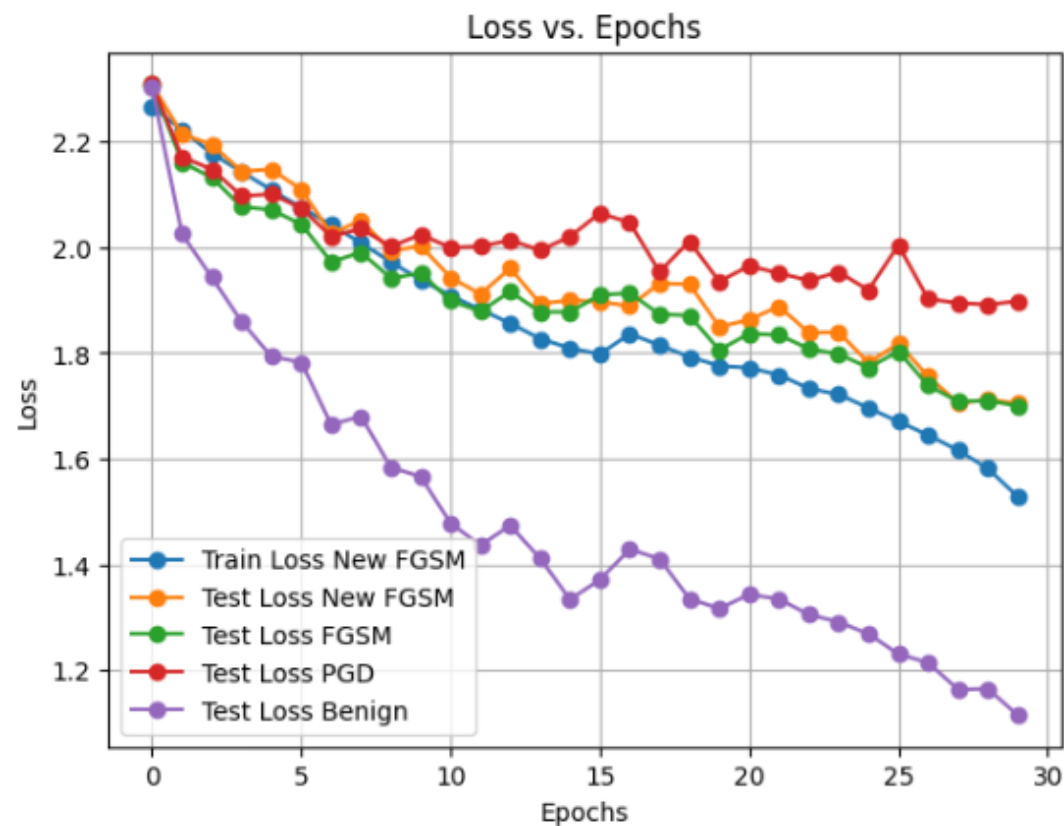
New FGSM Performance (CIFAR100)



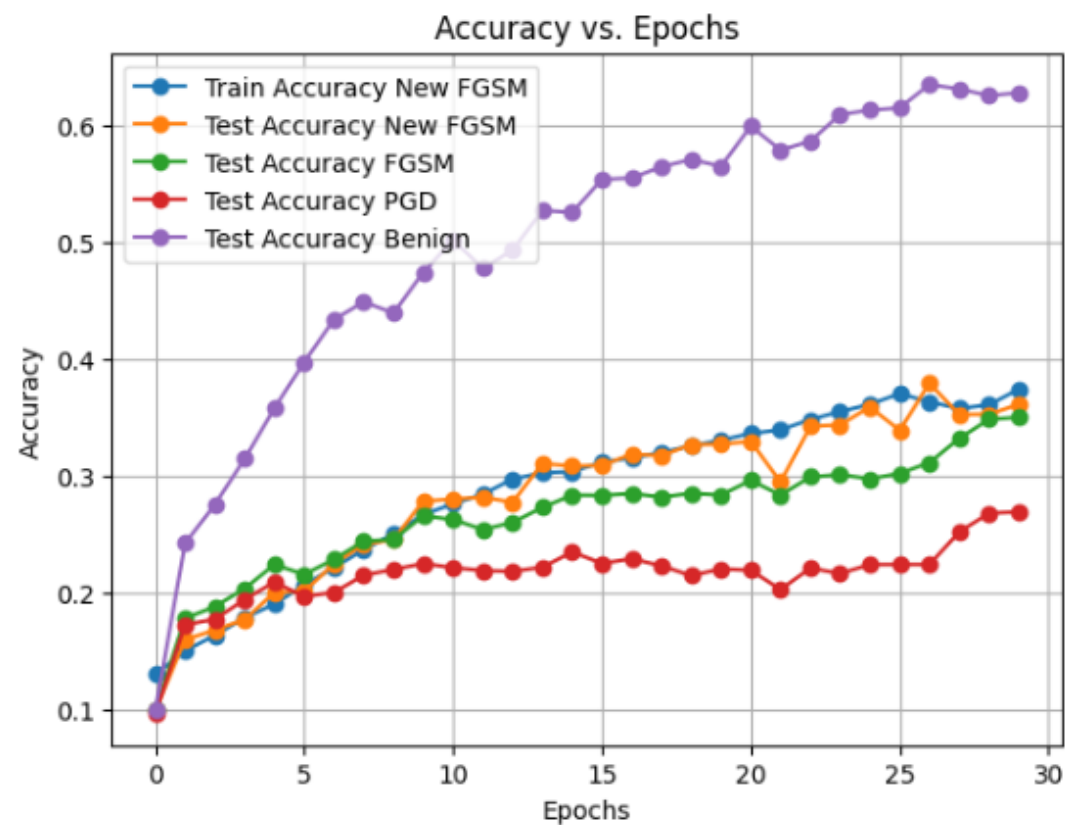
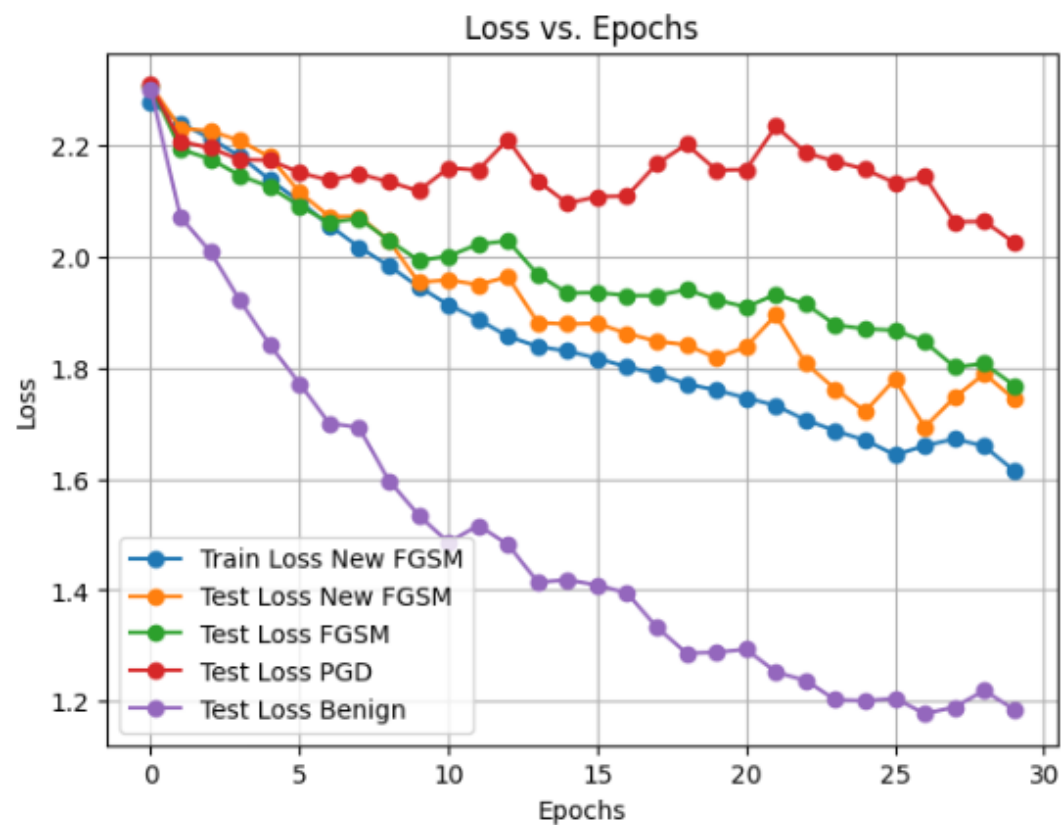
New FGSM Performance (Epsilon = 10/255)



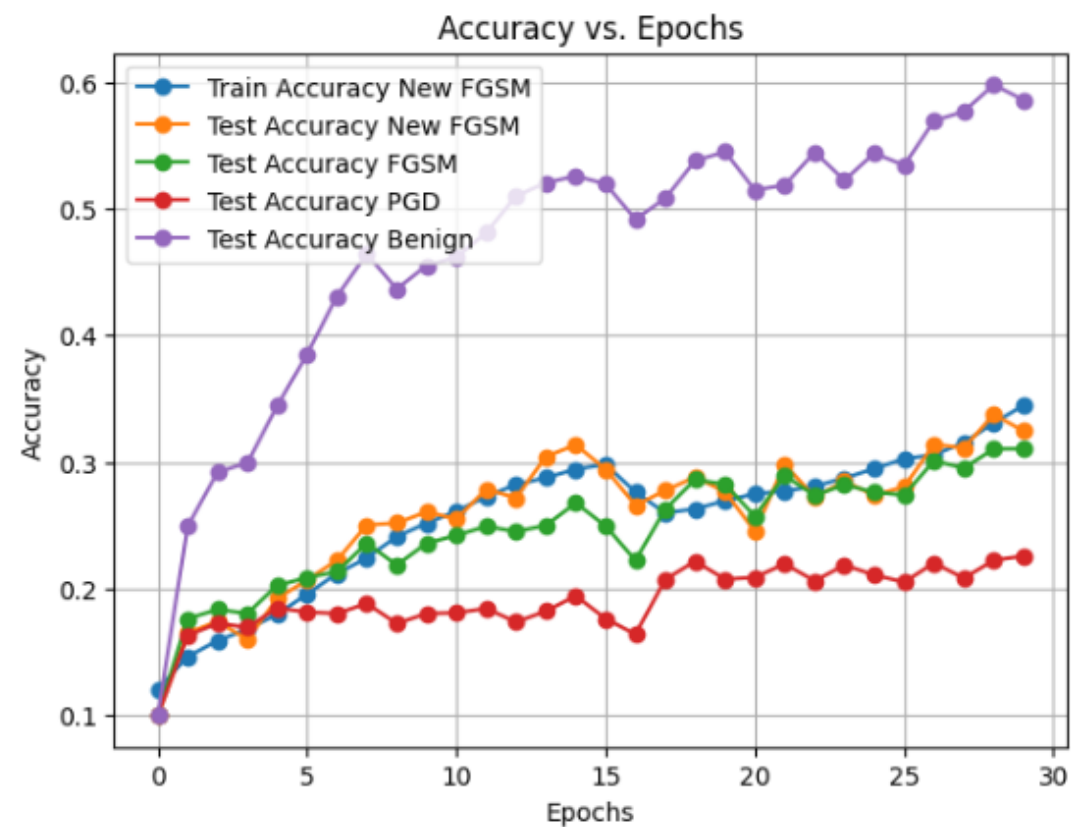
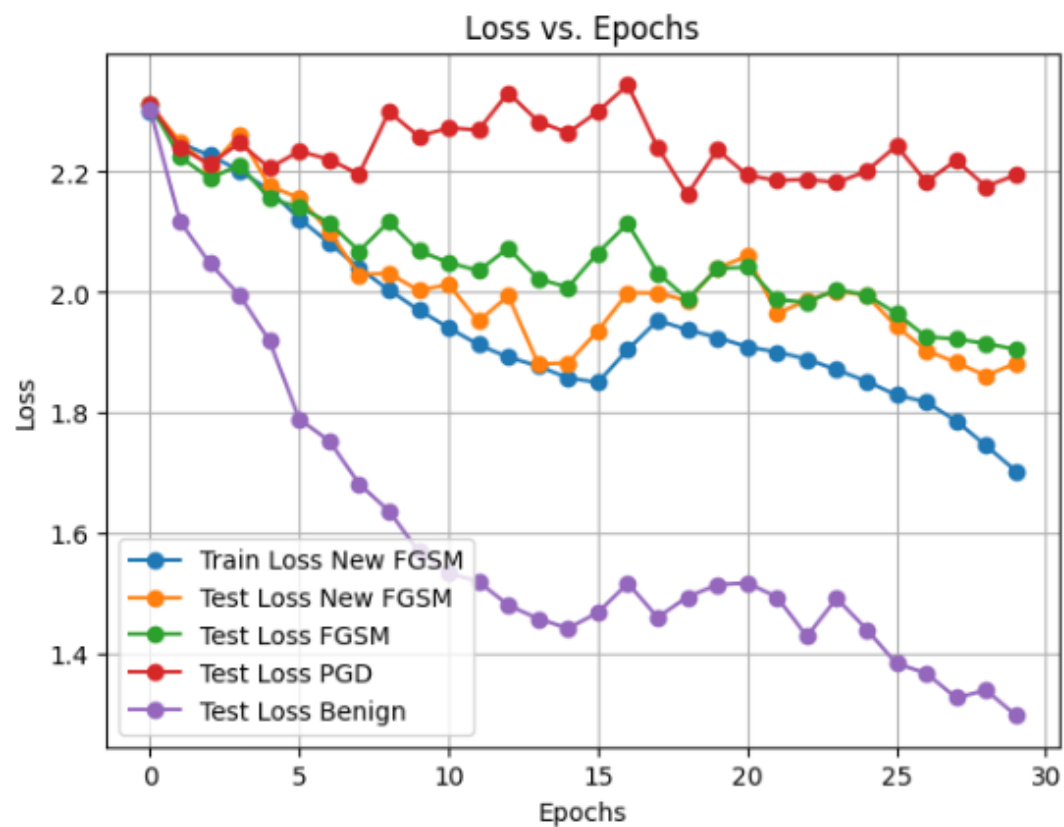
New FGSM Performance (Epsilon = 12/255)



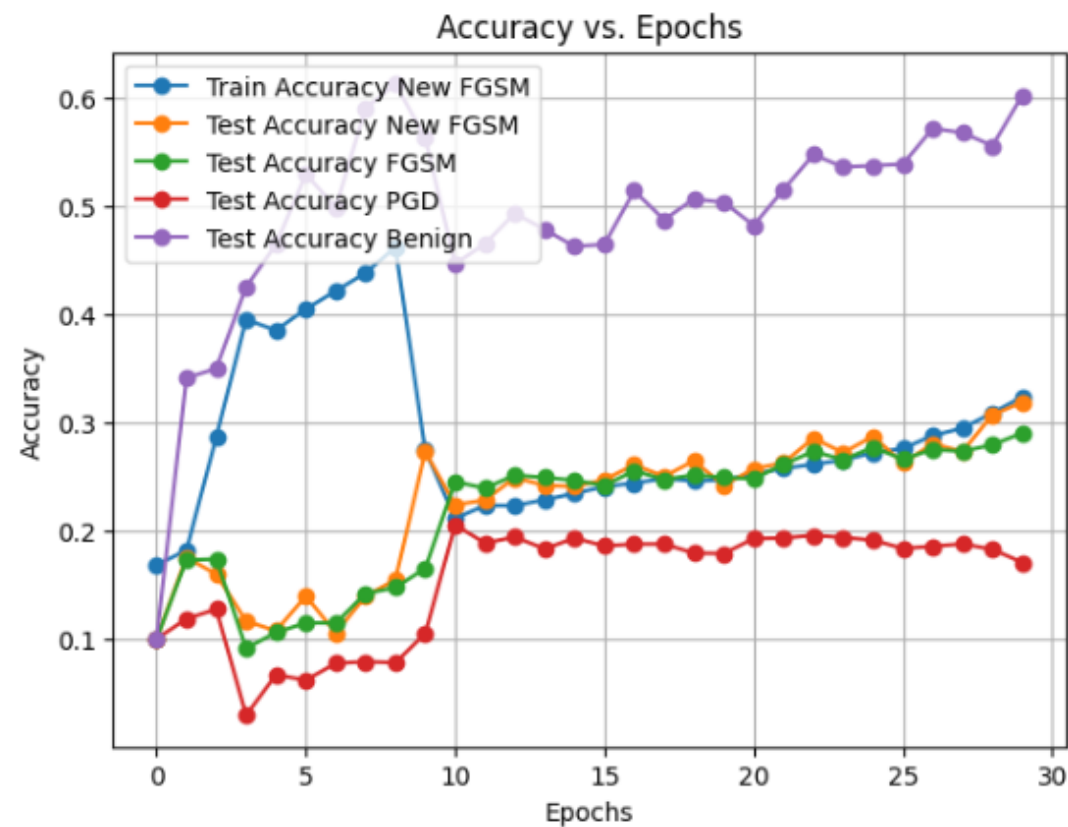
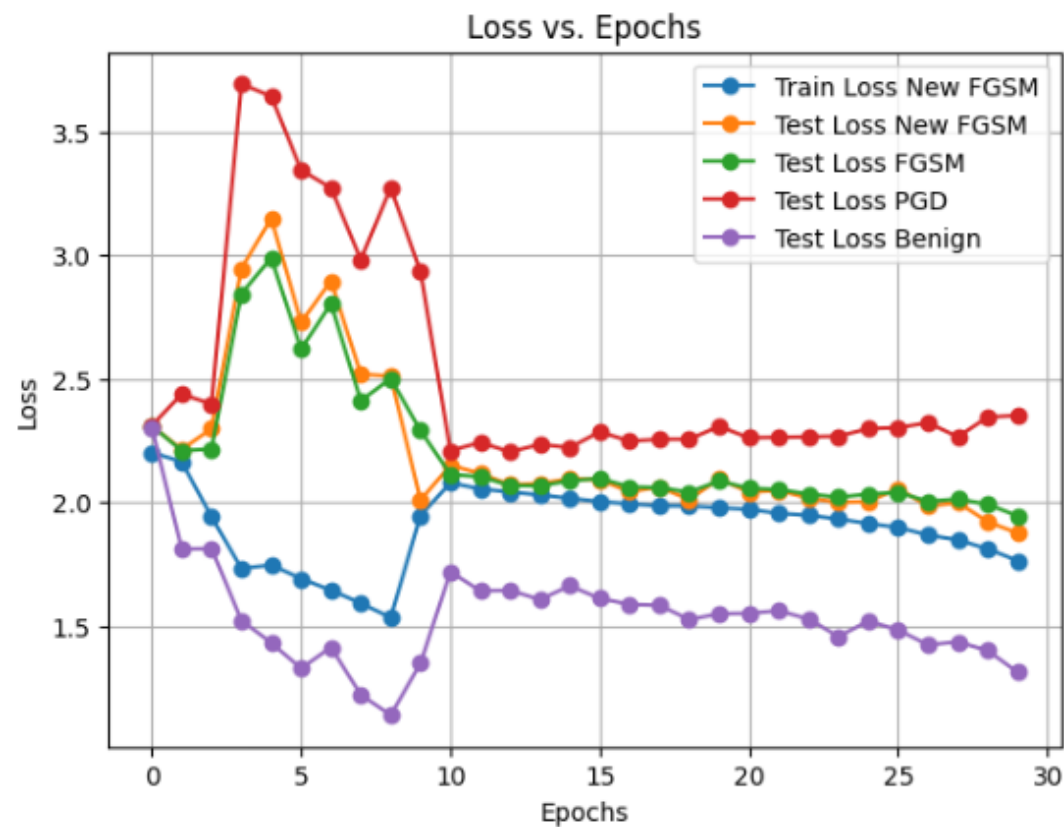
New FGSM Performance (Epsilon = 14/255)



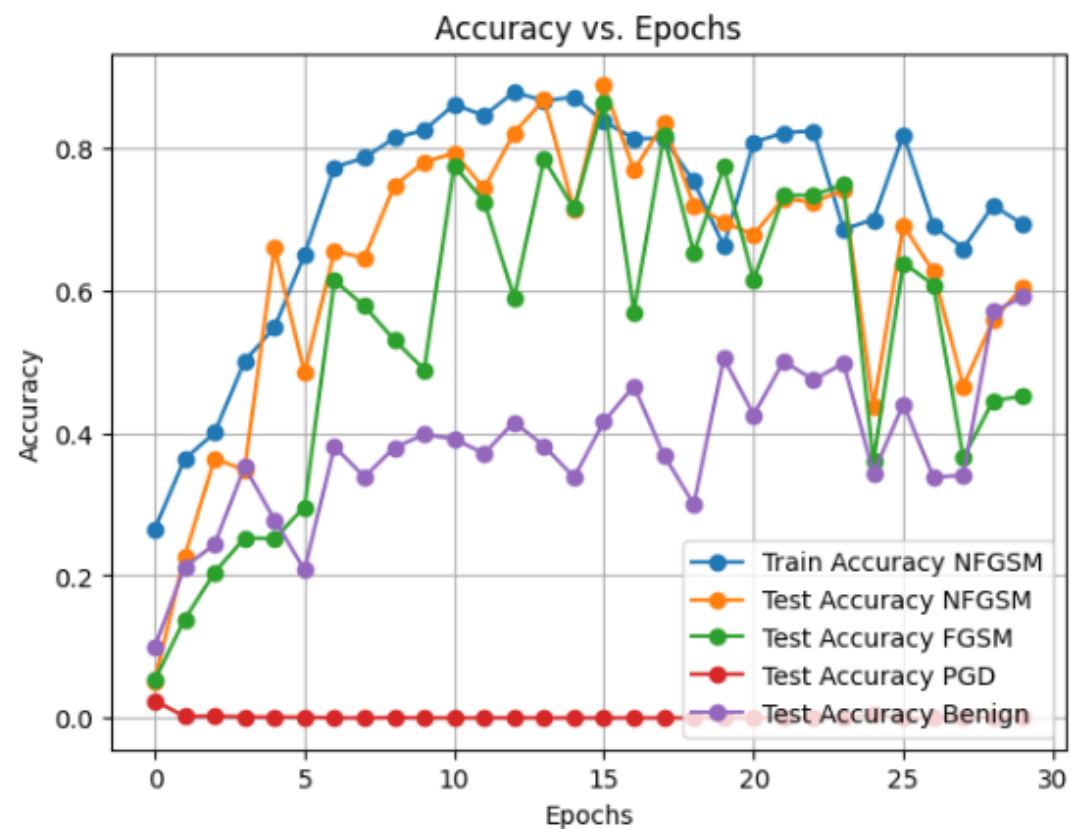
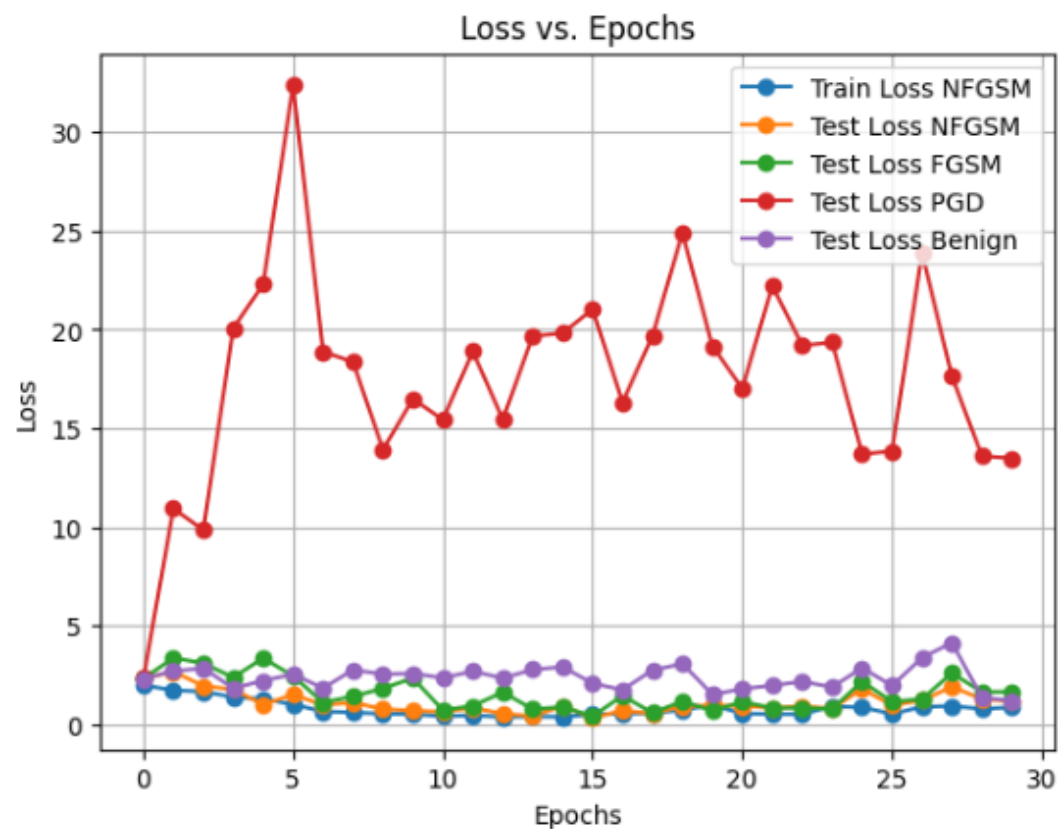
New FGSM Performance (Epsilon = 16/255)



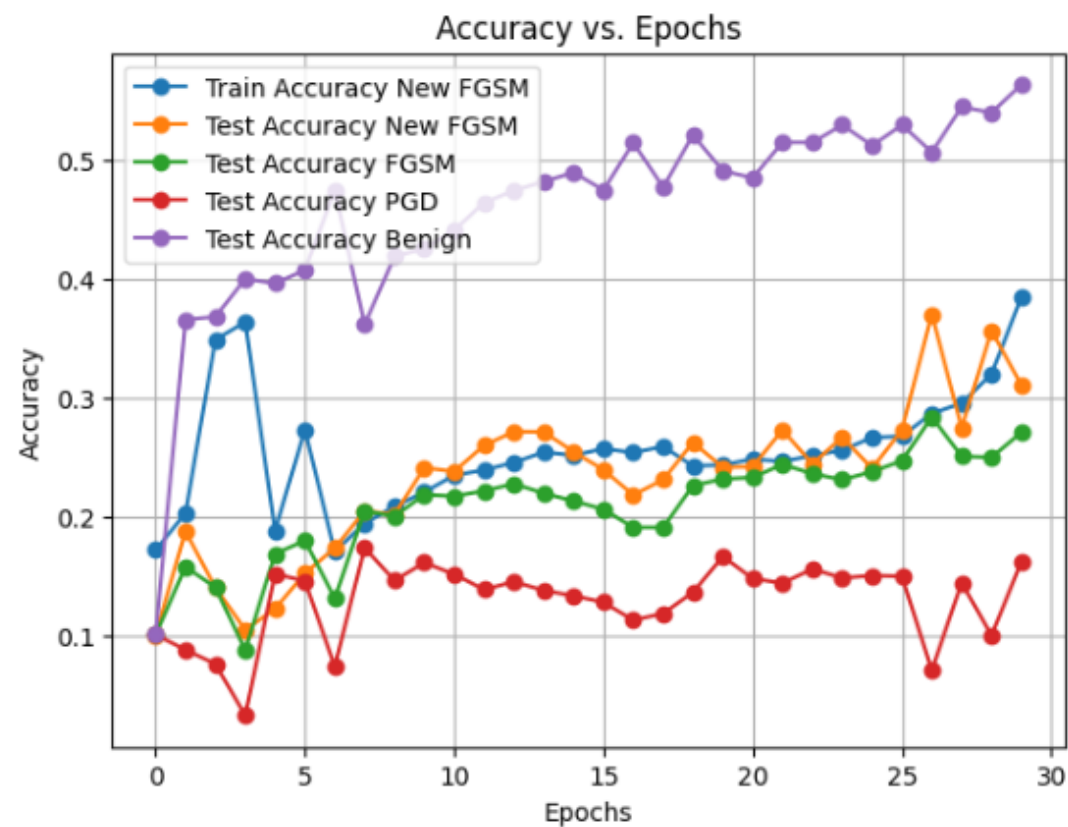
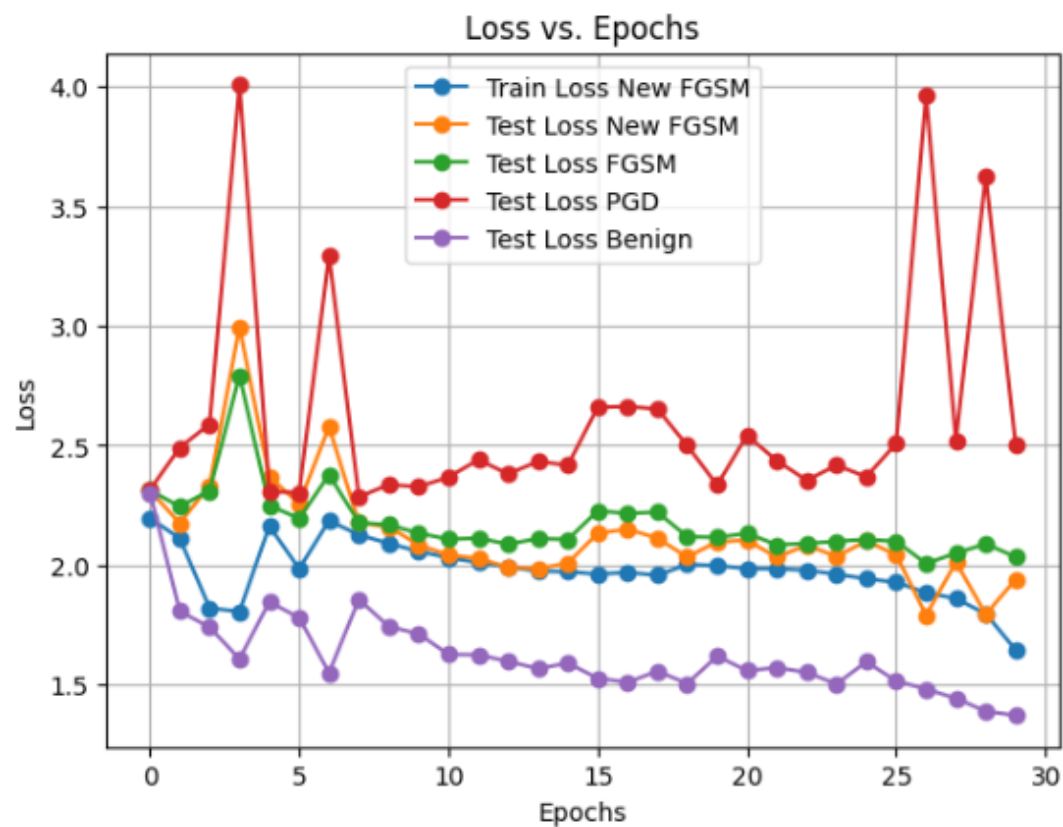
New FGSM Performance (Epsilon = 18/255)



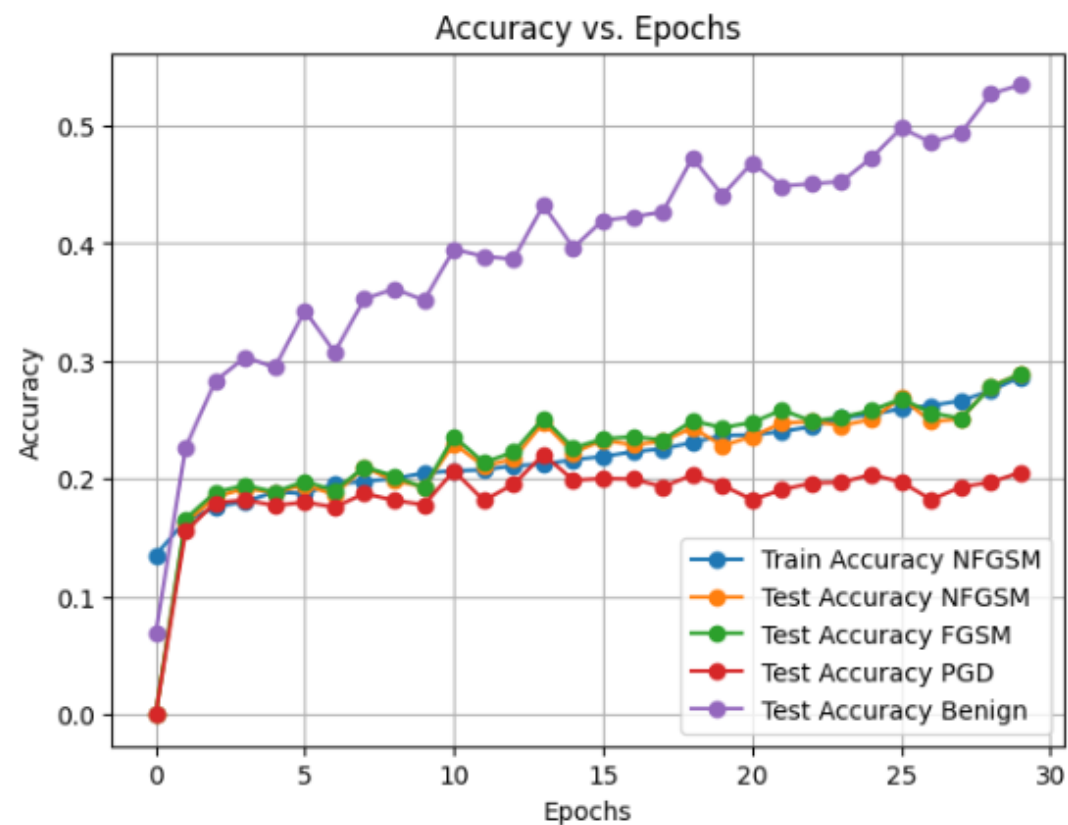
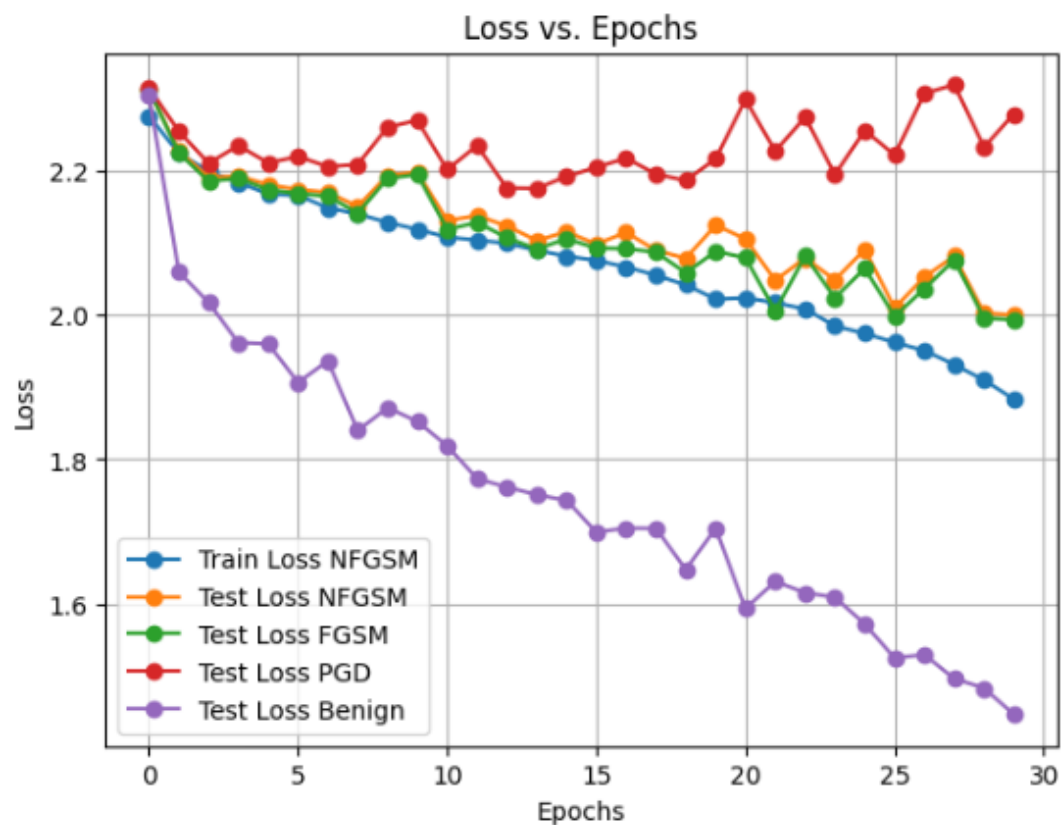
NFGSM Performance (Epsilon = 18/255)



New FGSM Performance (Epsilon = 20/255)

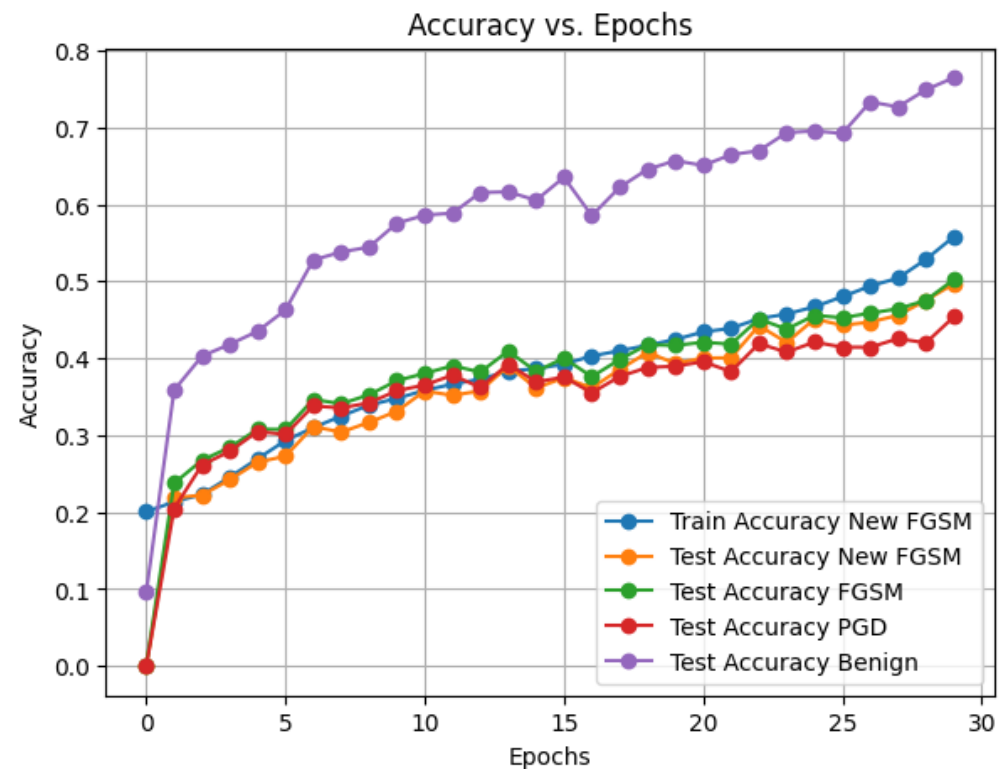
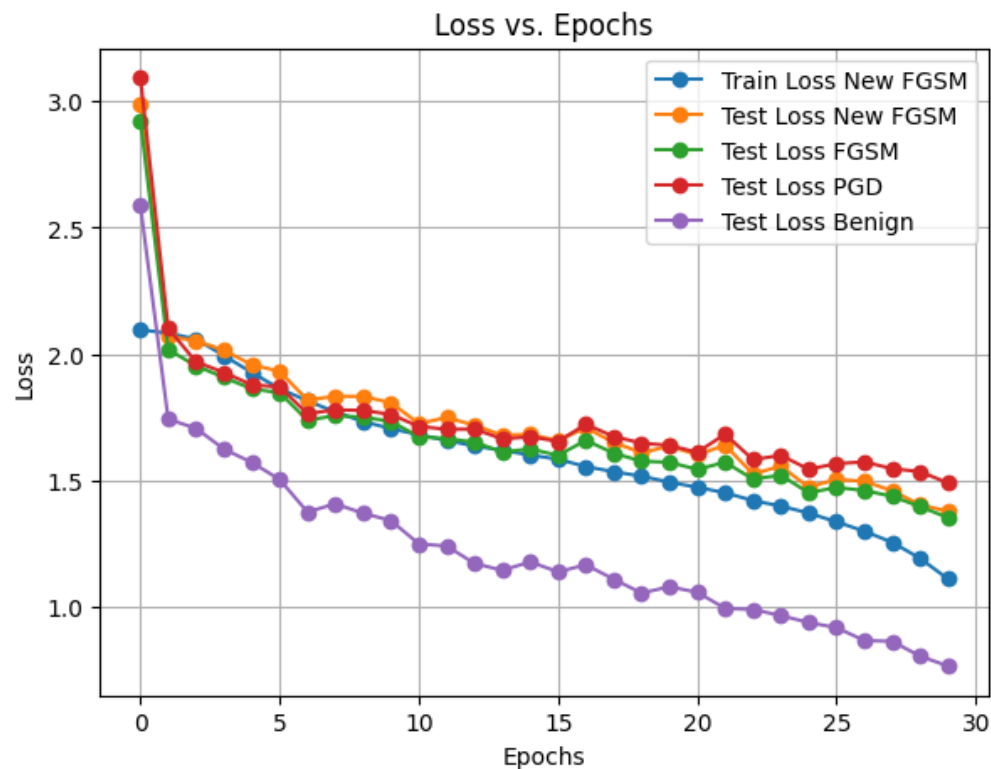


NFGSM Performance (Epsilon = 20/255)



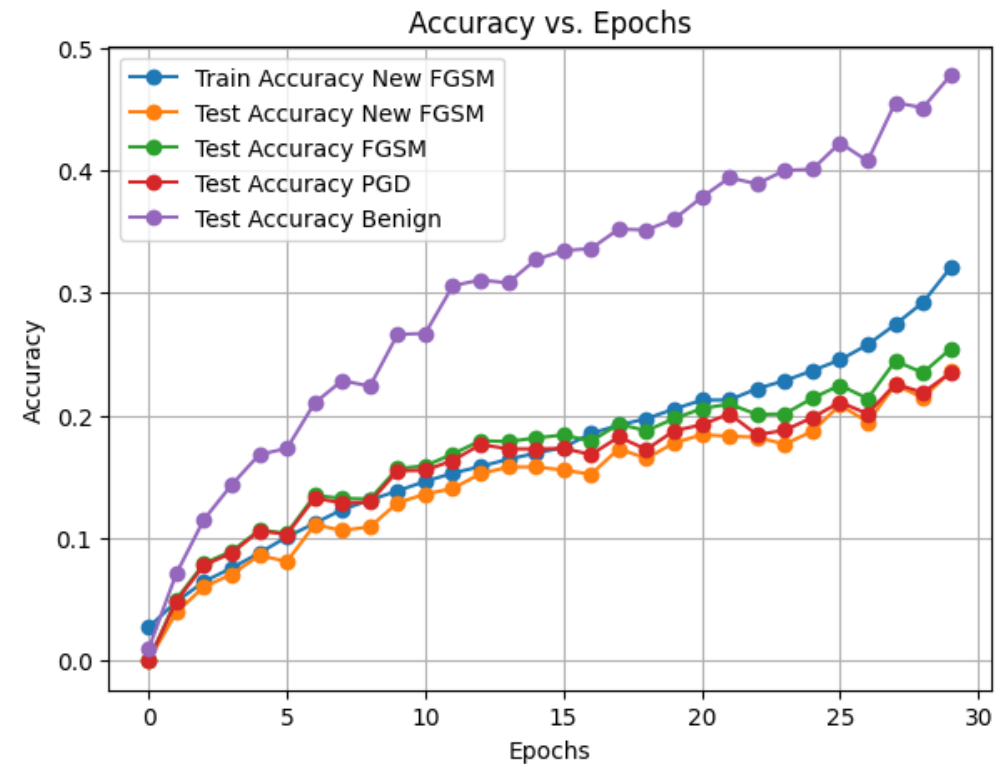
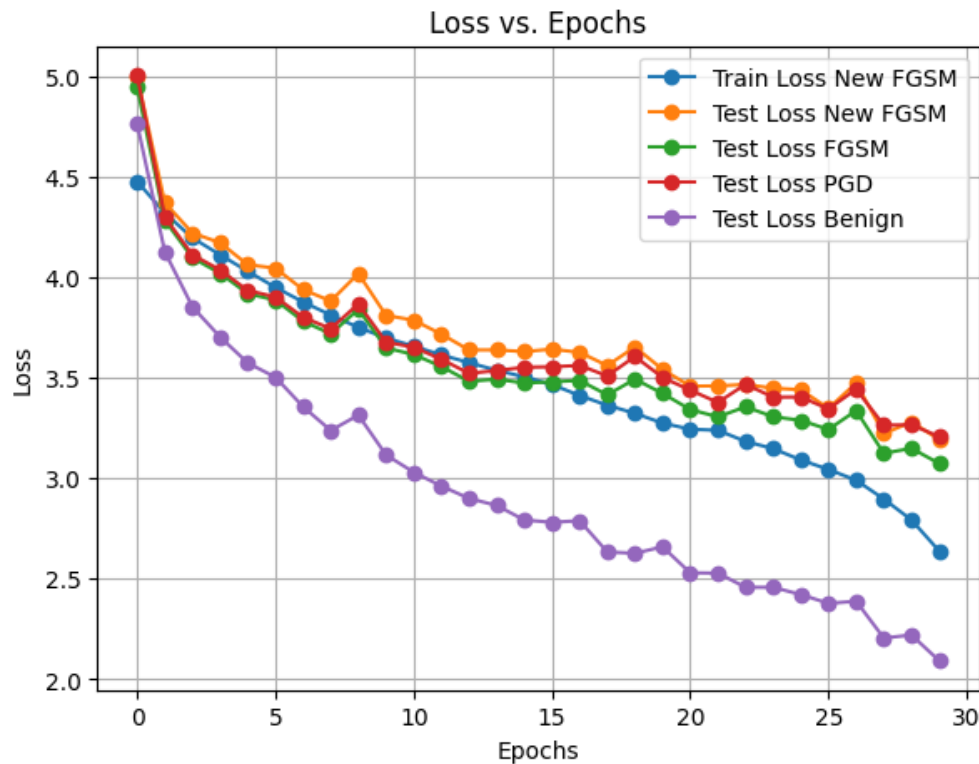
New FGSM Performance WideResNet(34, 10)

- CIFAR10



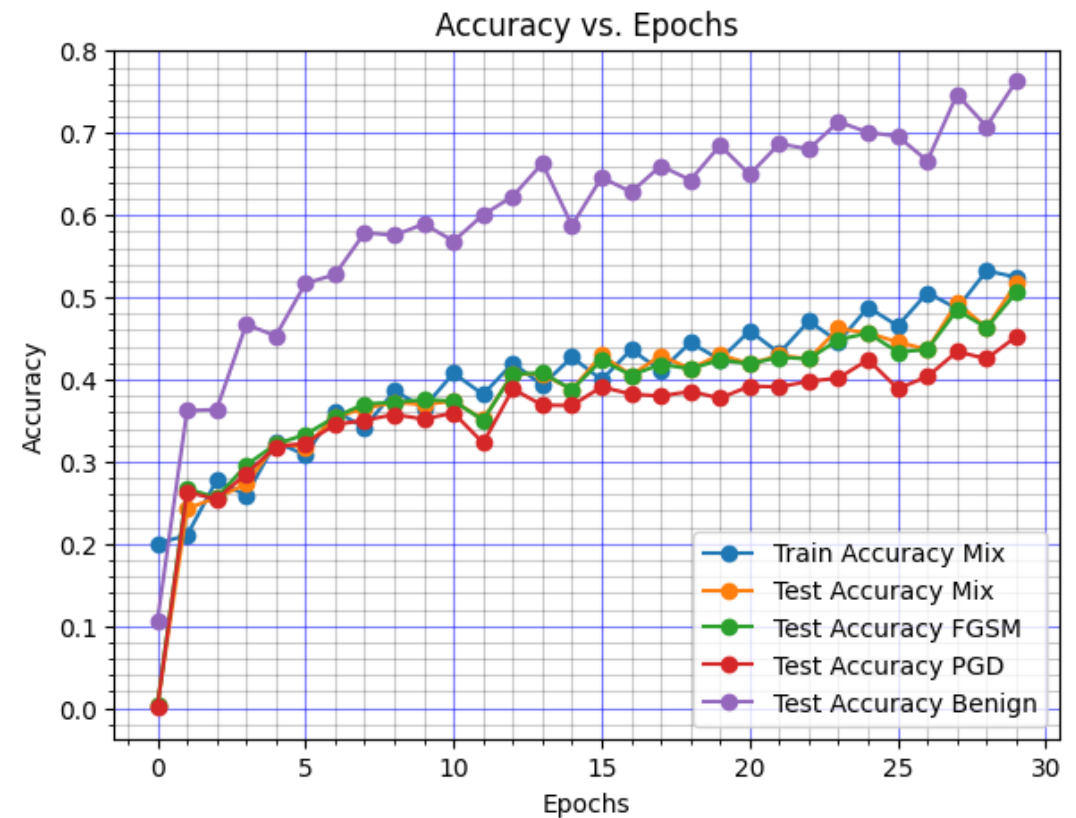
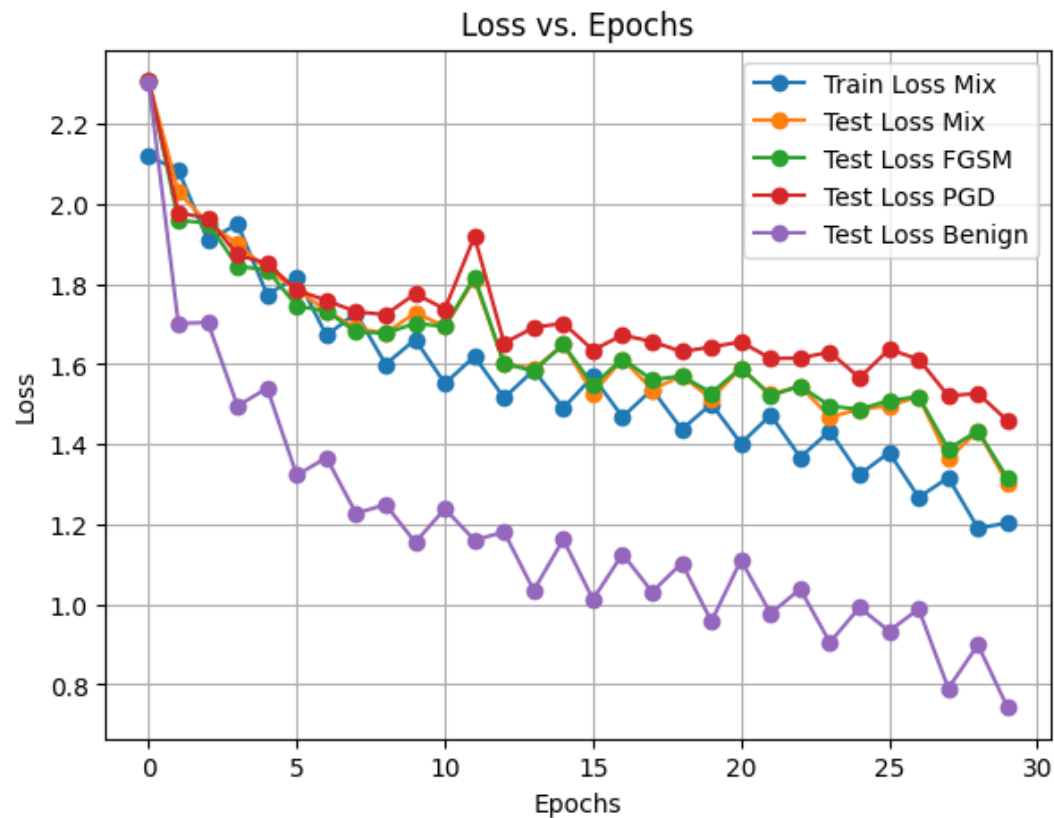
New FGSM Performance WideResNet(28, 10)

- CIFAR100

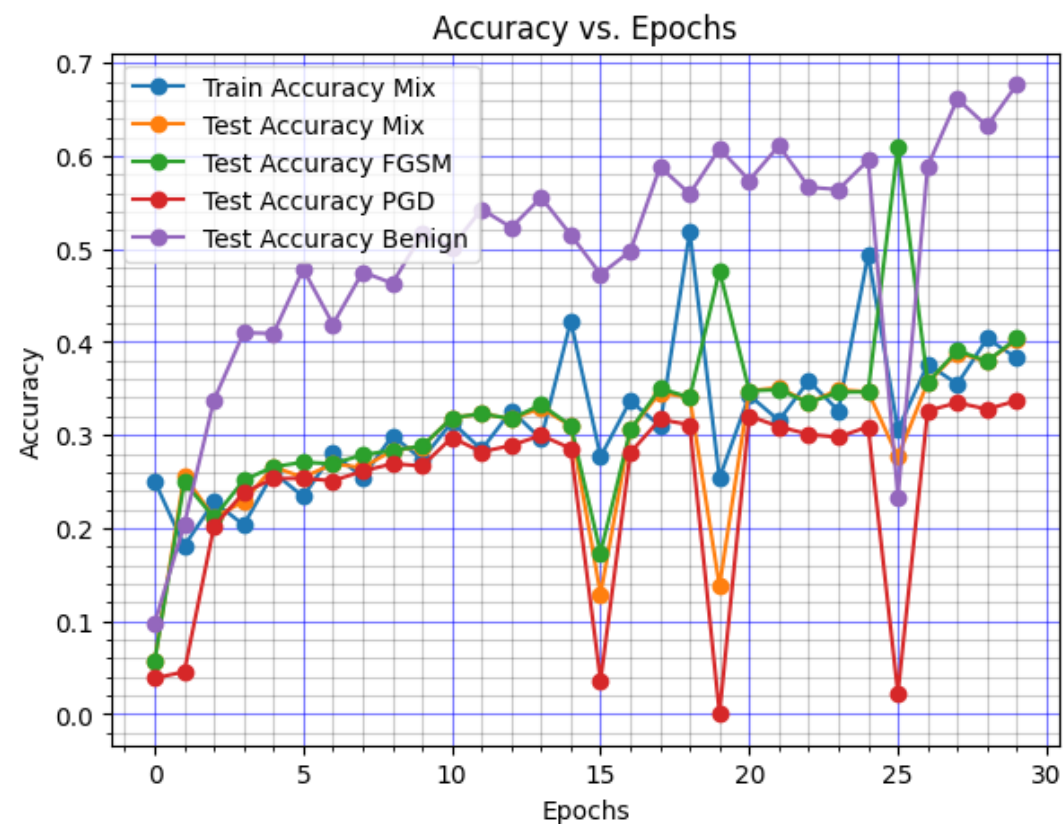
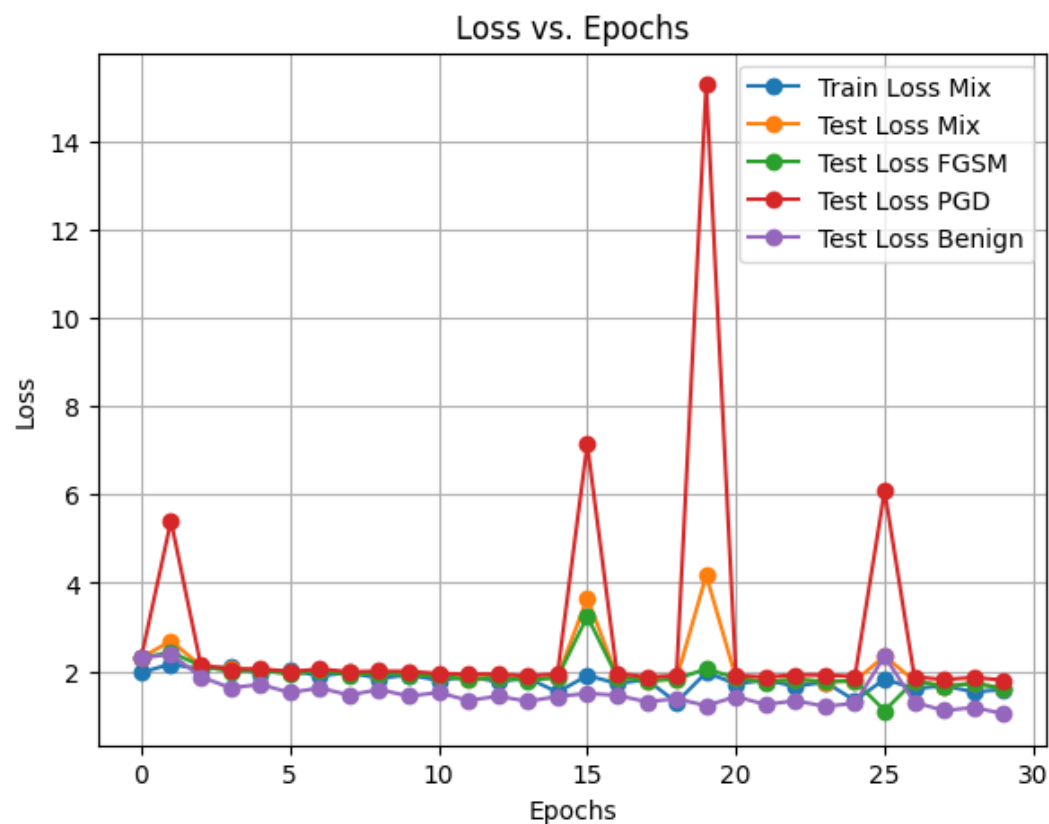


Mix Attack

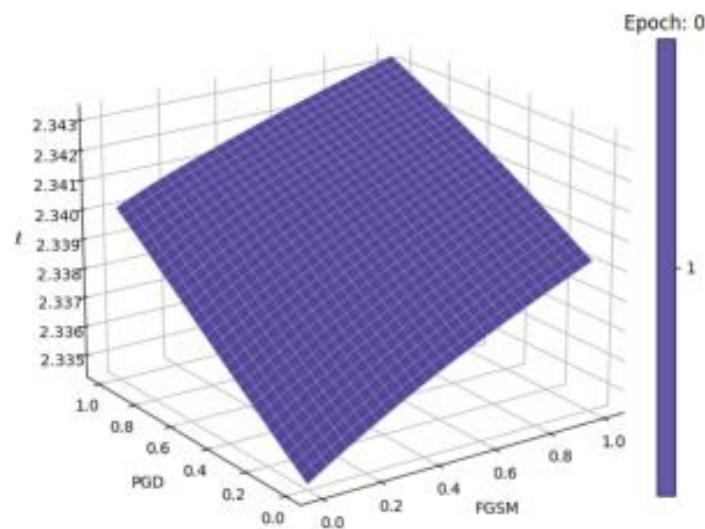
- Achieves NFGSM performance on $\epsilon = 8/255$



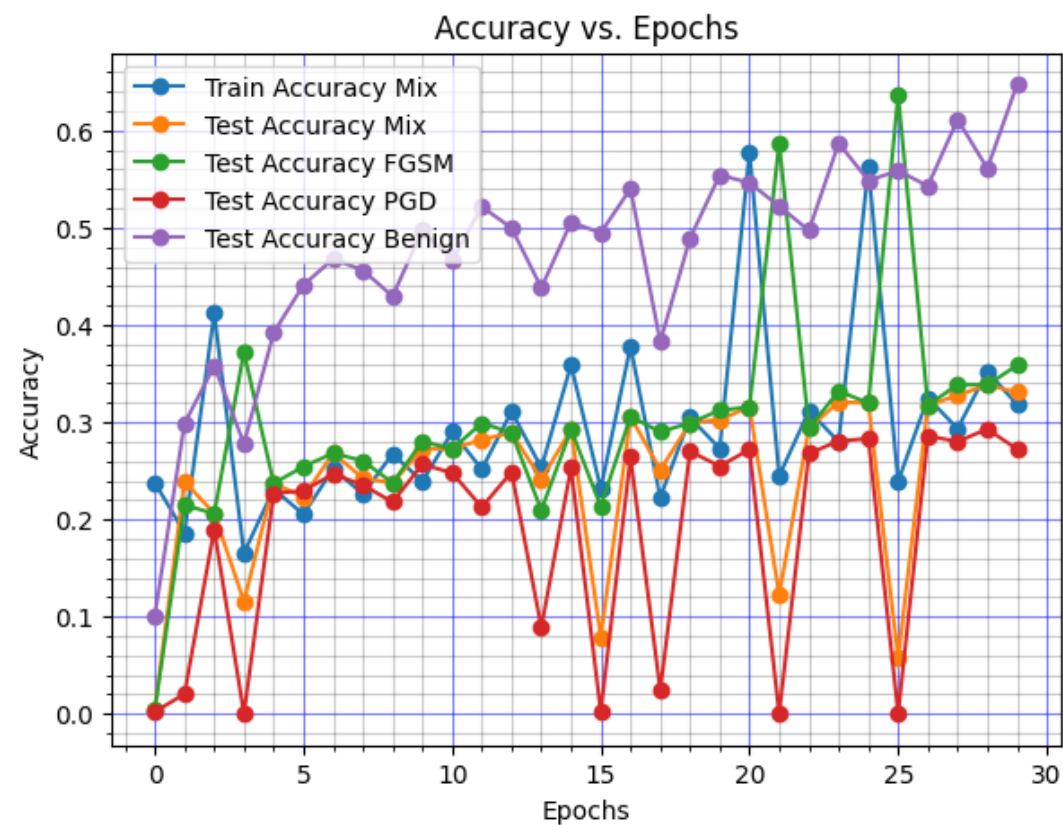
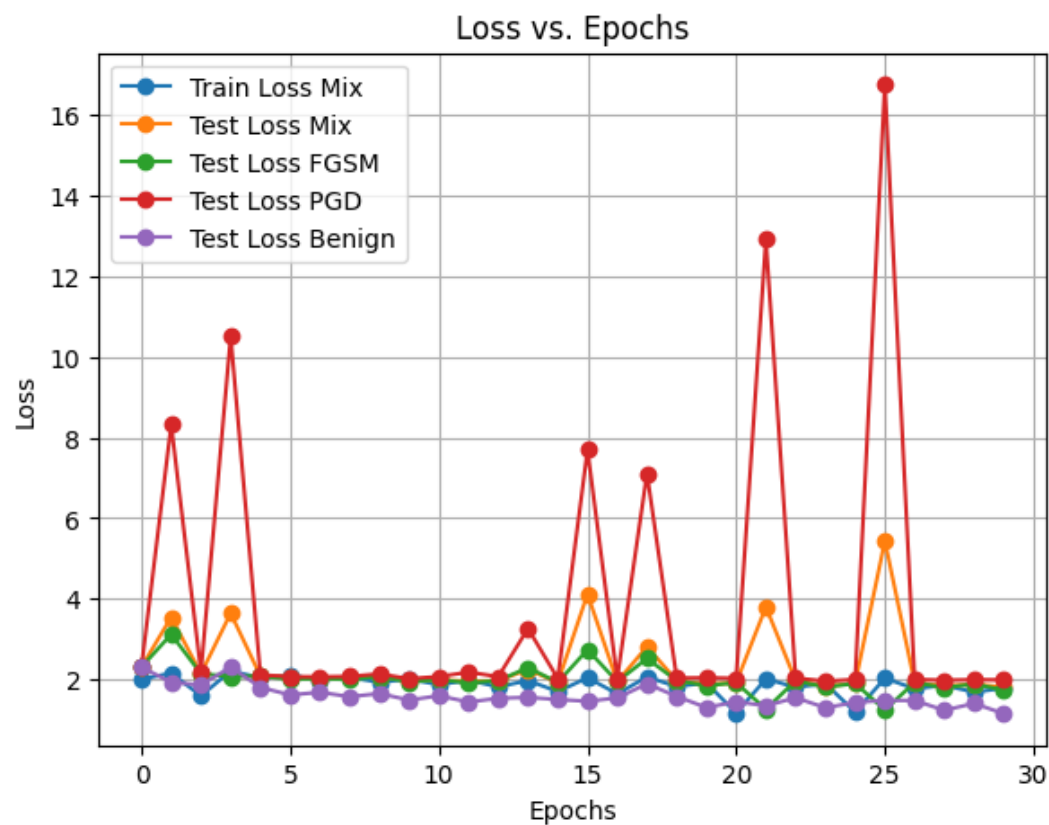
Mix Attack (Epsilon = 12/255)



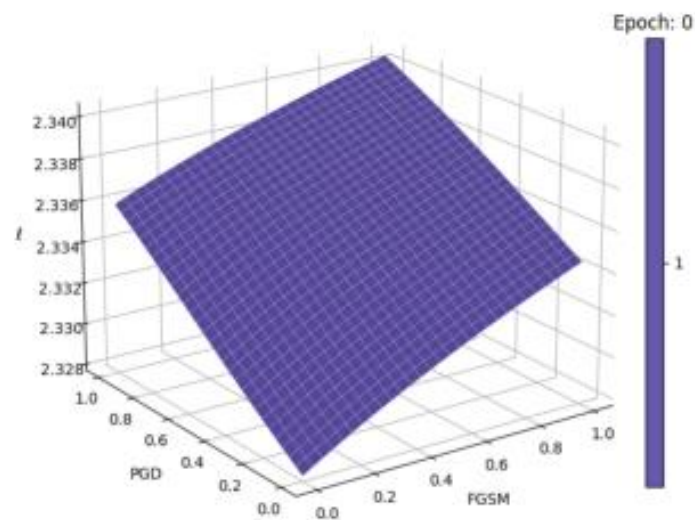
Mix Attack Loss Surface (Epsilon = 12/255)



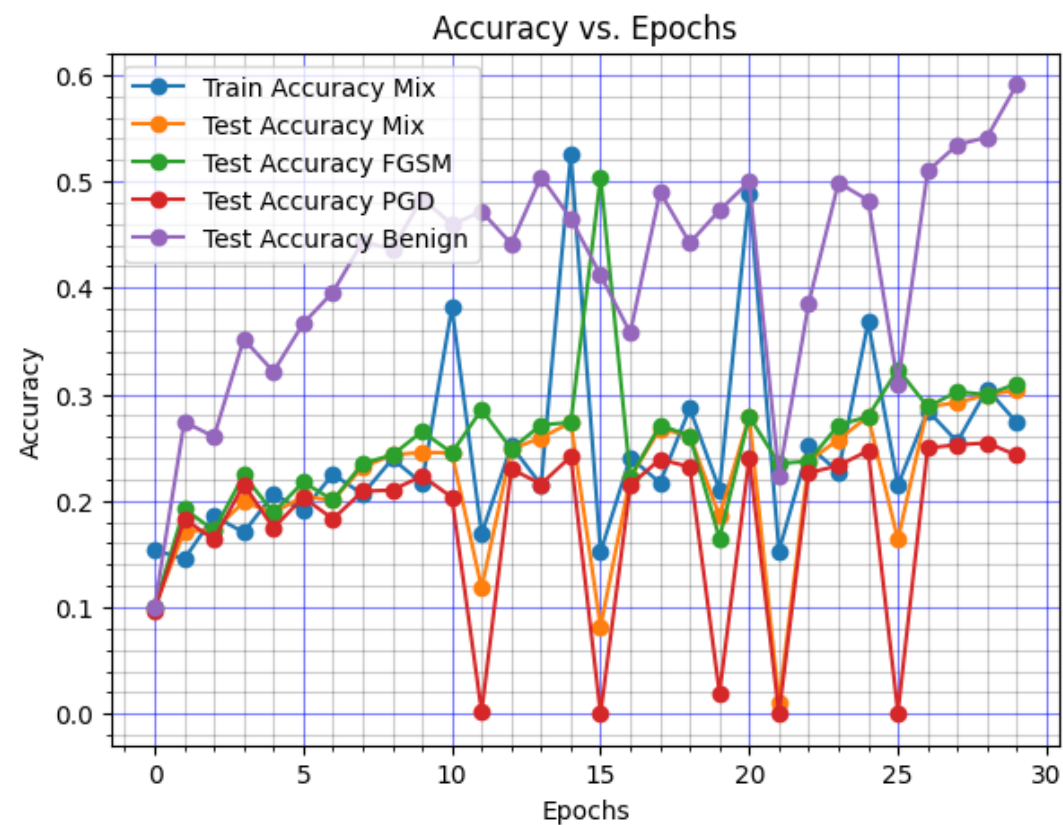
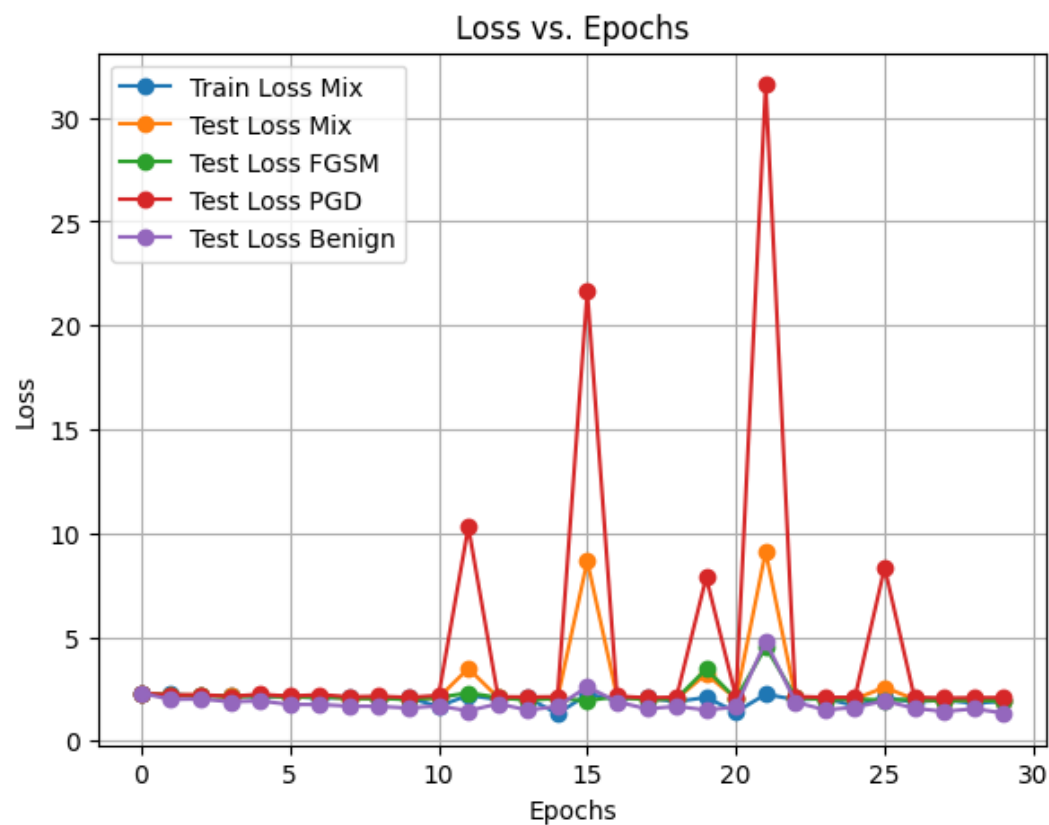
Mix Attack (Epsilon = 14/255)



Mix Attack Loss Surface (Epsilon = 14/255)



Mix Attack (Epsilon = 16/255)



Curv FGSM

- Uses idea from ELLE paper

Algorithm 1 ELLE (ELLE-A) adversarial training. Pseudo-code in teal is only run for ELLE-A. μ and σ represent the mean and standard deviation respectively.

```
1: Inputs: # epochs  $T$ , # batches  $M$ , radius  $\epsilon$ , regularization parameter  $\lambda$  and decay rate  $\gamma$ .
2:  $\text{err\_list} = []$ ,  $\lambda_{\max} = \lambda$ ,  $\lambda = 0$ 
3: for  $t = 1, \dots, T$  do
4:   for  $i = 1, \dots, M$  do
5:      $\mathbf{x}_{\text{FGSM}}^i = \mathbf{x}^i + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}^i} \mathcal{L}(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}^i), y^i))$  ▷ Standard FGSM attack
6:      $\mathbf{x}_a^i, \mathbf{x}_b^i \sim \mathbf{x}^i + \text{Unif}([- \epsilon, \epsilon]^d)$  ▷ Random samples
7:      $\alpha \sim \text{Unif}([0, 1])$ 
8:      $\mathbf{x}_c^i = (1 - \alpha) \cdot \mathbf{x}_a^i + \alpha \cdot \mathbf{x}_b^i$  ▷ Linear (convex) combination of  $\mathbf{x}_a^i$  and  $\mathbf{x}_b^i$ 
9:      $E_{\text{lin}} = |\mathcal{L}(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_c^i), y^i) - (1 - \alpha) \cdot \mathcal{L}(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_a^i), y^i) - \alpha \cdot \mathcal{L}(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_b^i), y^i)|^2$ 
10:    if  $E_{\text{lin}} > \mu(\text{err\_list}) + 2 \cdot \sigma(\text{err\_list})$  then  $\lambda = \lambda_{\max}$ 
11:    else  $\lambda = \gamma \cdot \lambda$ 
12:     $\text{err\_list} = \text{err\_list} + [E_{\text{lin}}]$ 
13:     $\nabla_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_{\text{FGSM}}^i), y^i) + \lambda \nabla_{\boldsymbol{\theta}} E_{\text{lin}}$  ▷ Compute parameter gradients
14:     $\boldsymbol{\theta} = \text{optimizer}(\boldsymbol{\theta}, \nabla_{\boldsymbol{\theta}})$  ▷ Standard parameters update, (e.g. SGD)
```

Curv FGSM

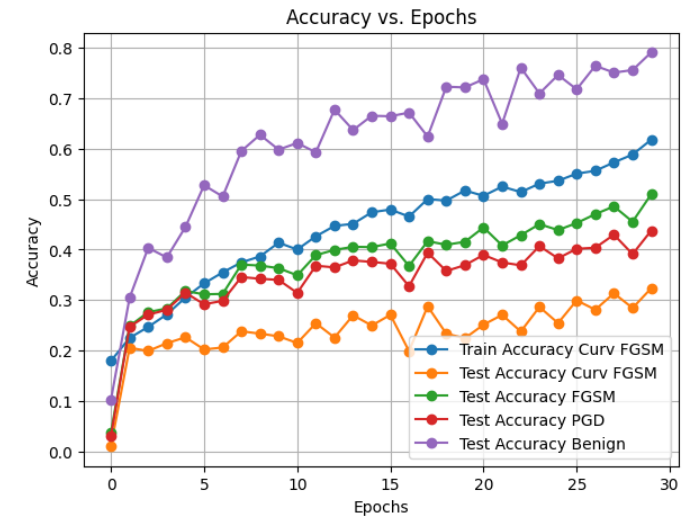
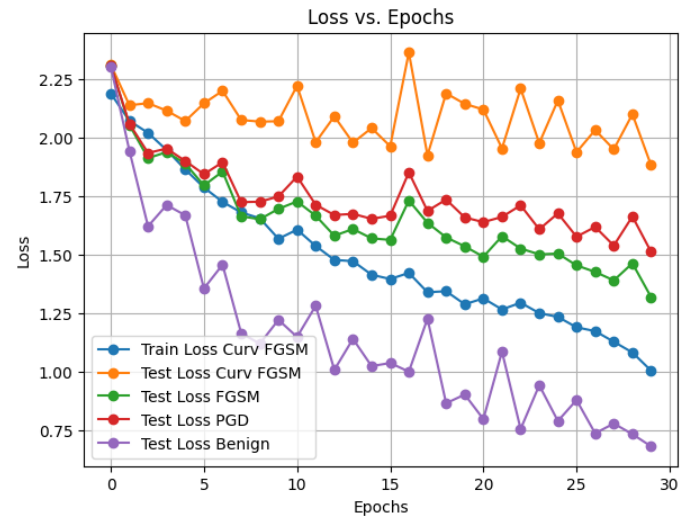
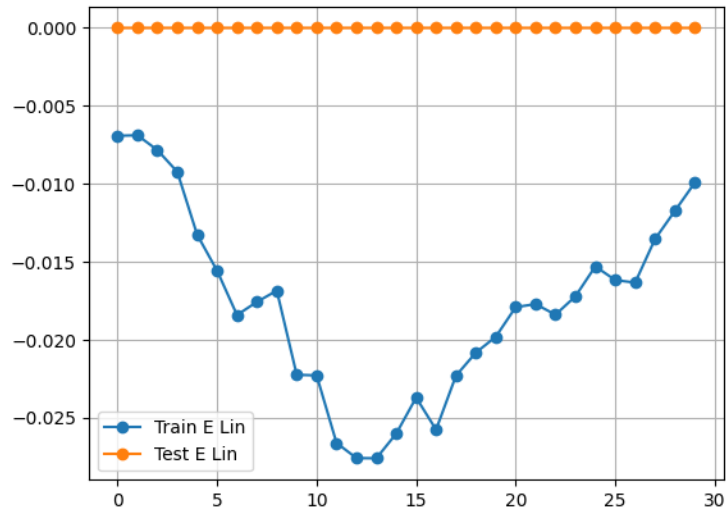
```
# if e_lin < 0 -> concave
# if e_lin > 0 -> convex
# if e_lin = 0 -> linear
def alpha_from_curv(x, y, model, epsilon, alpha, loss_start, grad):
    model_is_training = model.training
    if model_is_training:
        model.eval()
    with torch.no_grad():
        x = x.clone().detach().cuda()
        y = y.clone().detach().cuda()
        grad = grad.clone().detach().cuda()
        x_end = x + alpha * grad
        theta = torch.rand(1)[0]
        x_between = theta * x + (1 - theta) * x_end
        loss = nn.CrossEntropyLoss()
        preds_end = model(x_end)
        loss_end = loss(preds_end, y)
        preds_between = model(x_between)
        loss_between = loss(preds_between, y)
        e_lin = theta * loss_start + (1 - theta) * loss_end.item() - loss_between.item()

    if model_is_training:
        model.train()
    return e_lin
```

Curv FGSM

```
delta.requires_grad = True
output = model(x + delta[:x.size(0)])
loss = F.cross_entropy(output, y)
loss.backward()
grad = delta.grad.detach()
#####
e_lin = alpha_from_curv(x, y, model, epsilon, alpha, loss.item(), grad)
if e_lin < -1e-4:
    alpha /= 3
elif e_lin < -1e-5:
    alpha /= 2.25
elif e_lin < -1e-6:
    alpha /= 2
elif e_lin < -1e-7:
    alpha /= 1.5
elif e_lin > 1e-4:
    alpha *= 1.25
#####
delta = delta + alpha * torch.sign(grad)
fgsm_x = x + delta
if max_range_clamp:
    fgsm_x = torch.clamp(x + delta, min=lower_limit, max=upper_limit).detach()
return fgsm_x, e_lin
```

Curv FGSM Performance

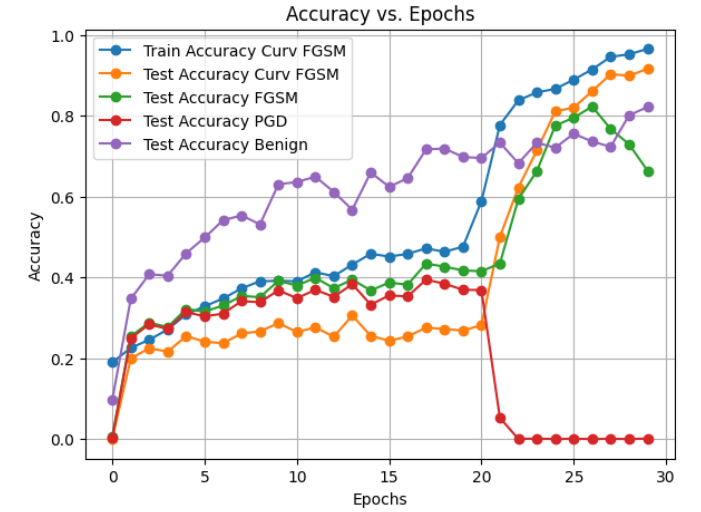
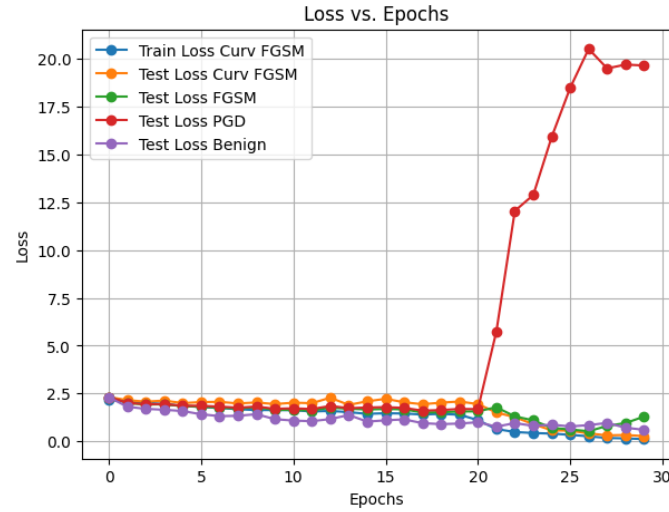
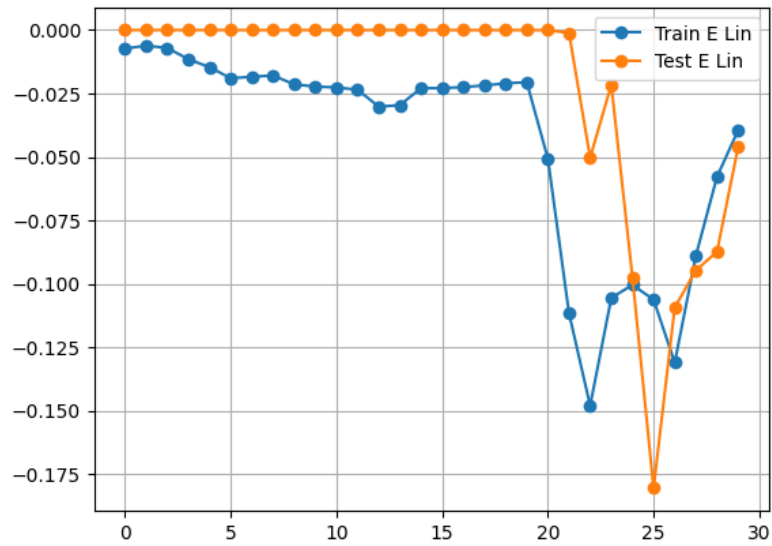


Curv FGSM Performance

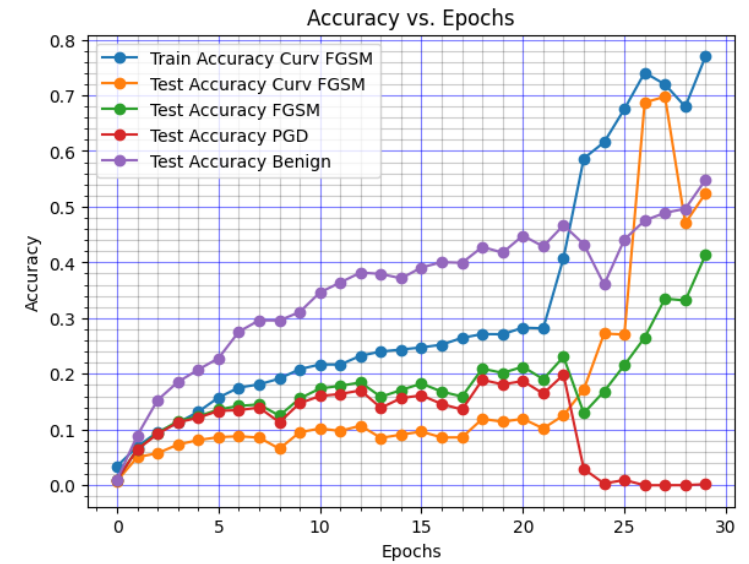
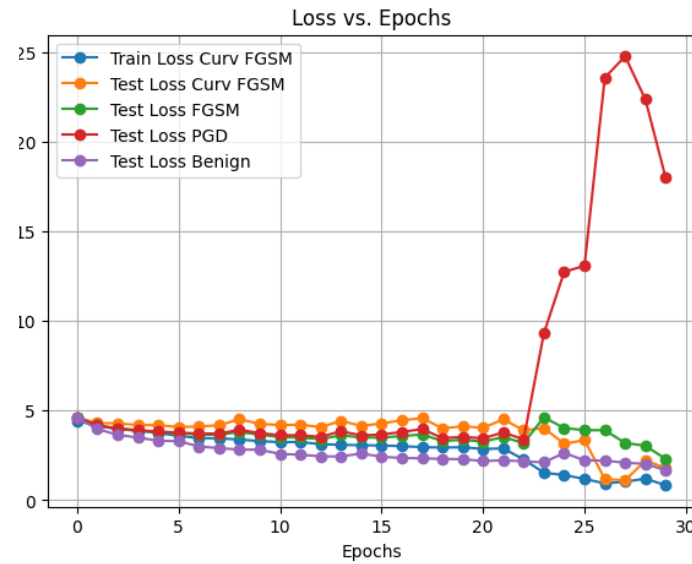
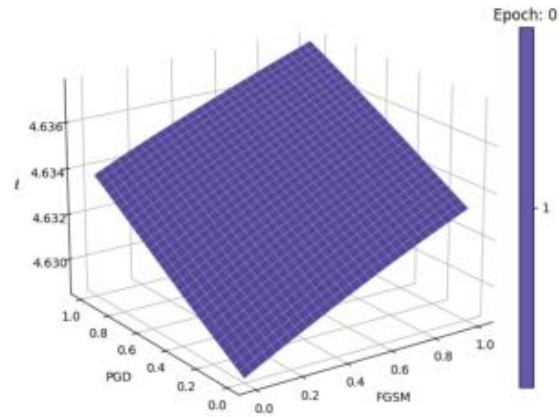
- Very sensitive to alpha function

```
if e_lin < -1e-3:  
    alpha /= 2.75  
elif e_lin < -1e-4:  
    alpha /= 2.5  
elif e_lin < -1e-5:  
    alpha /= 2.25  
elif e_lin < -1e-7:  
    alpha /= 2  
elif e_lin > 1e-4:  
    alpha *= 1.25
```

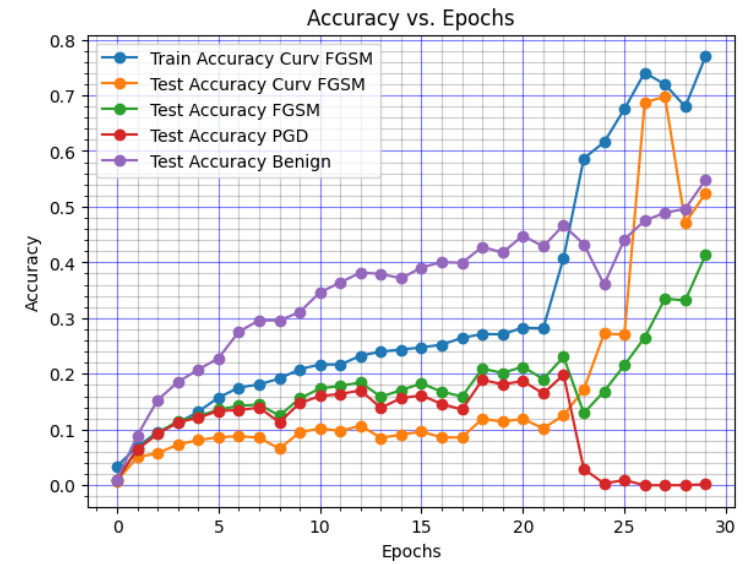
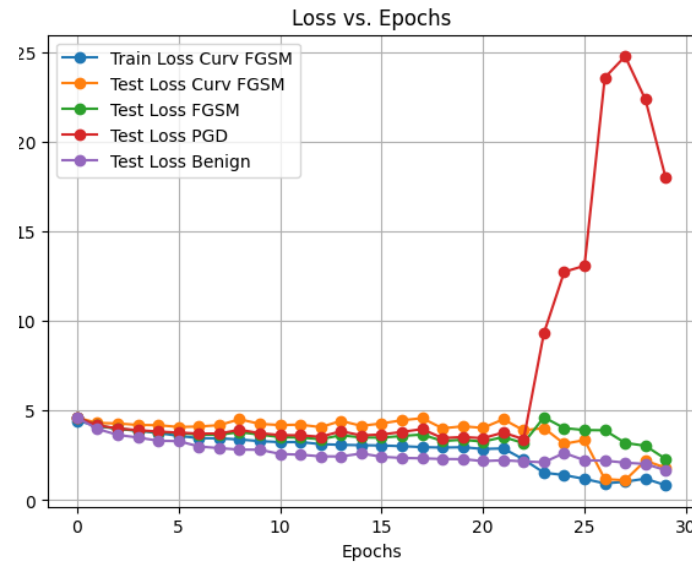
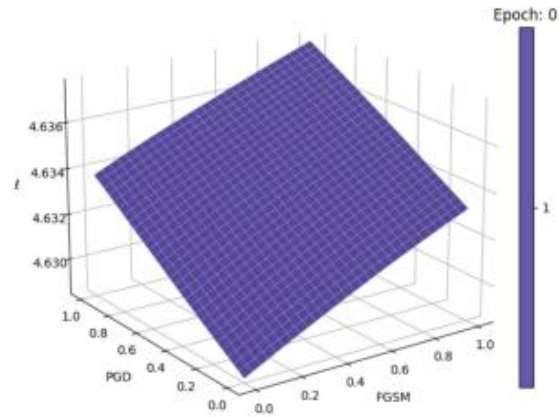
Curv FGSM Performance



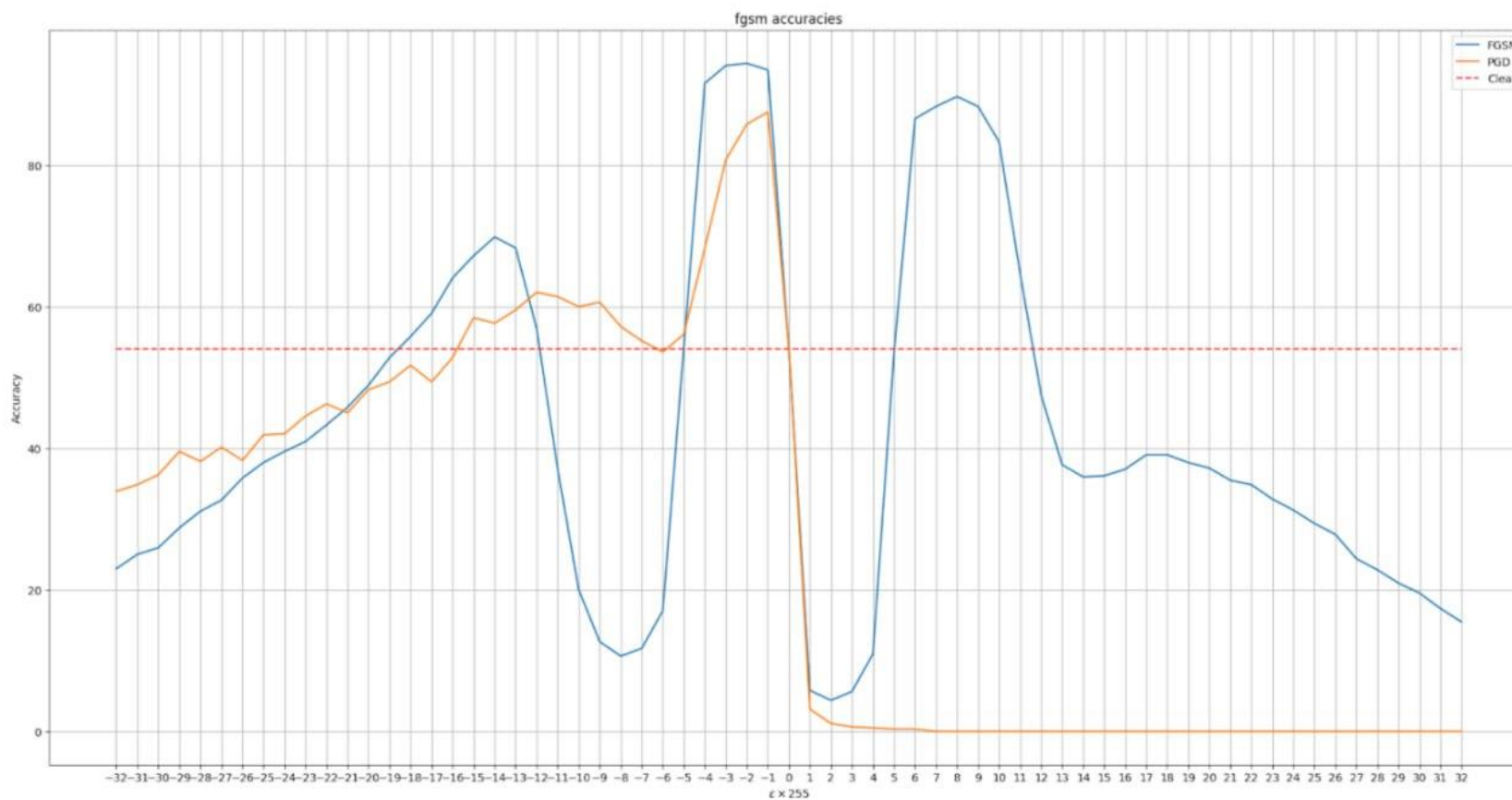
Curv FGSM Performance (CIFAR100)



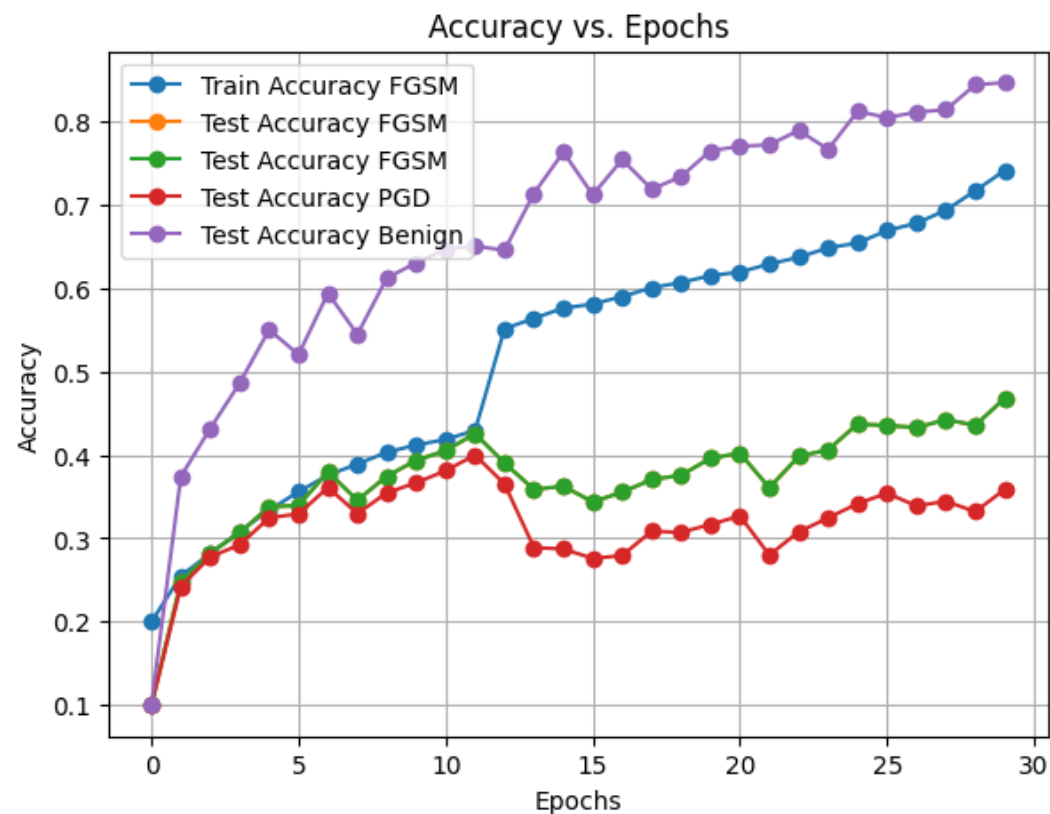
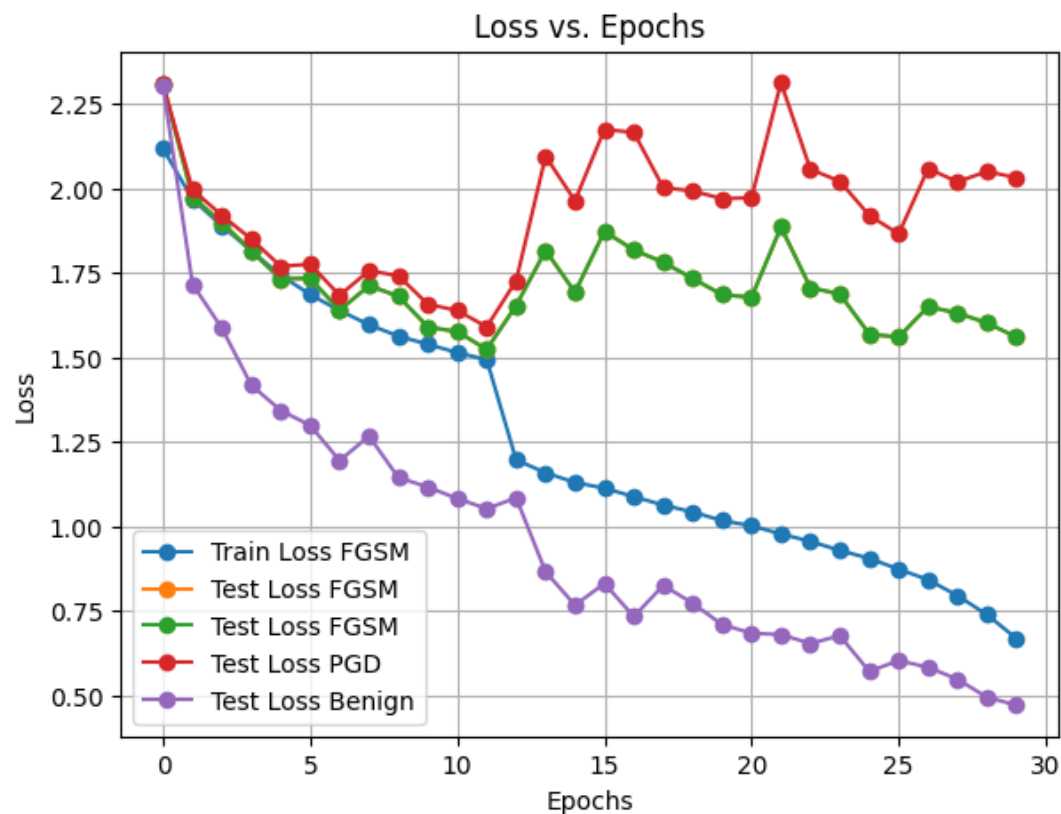
Curv FGSM Performance (CIFAR100)



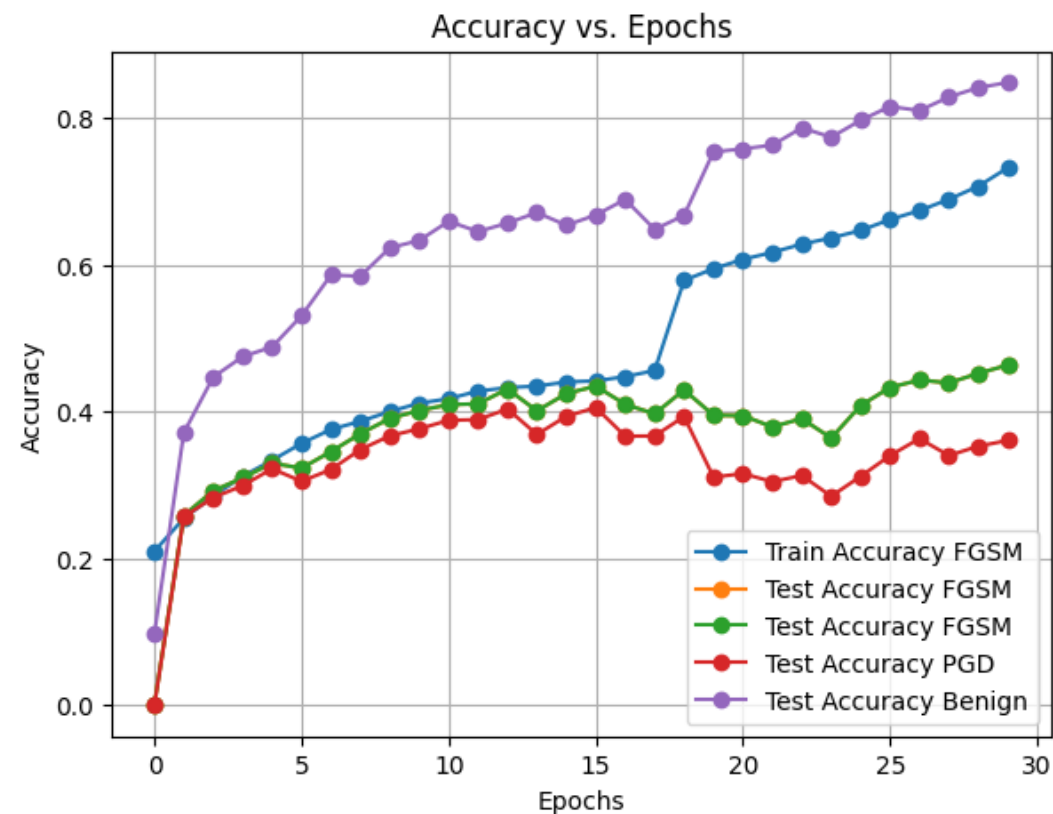
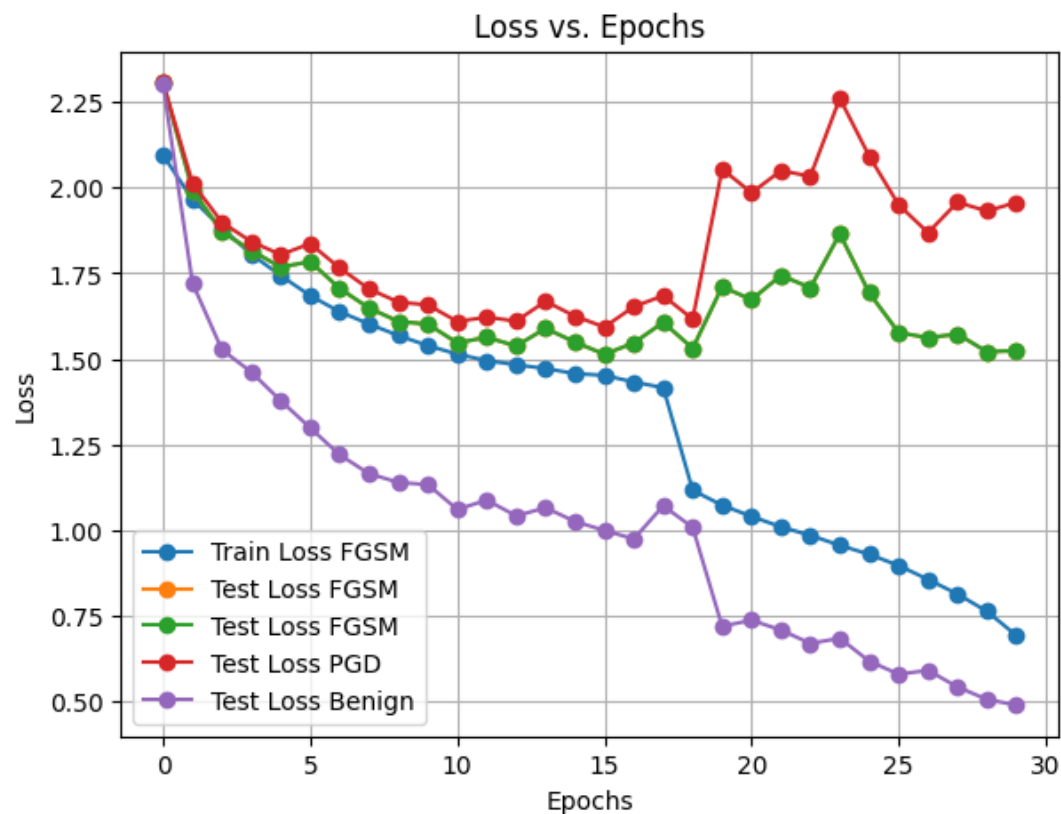
Effect of Alpha After CO



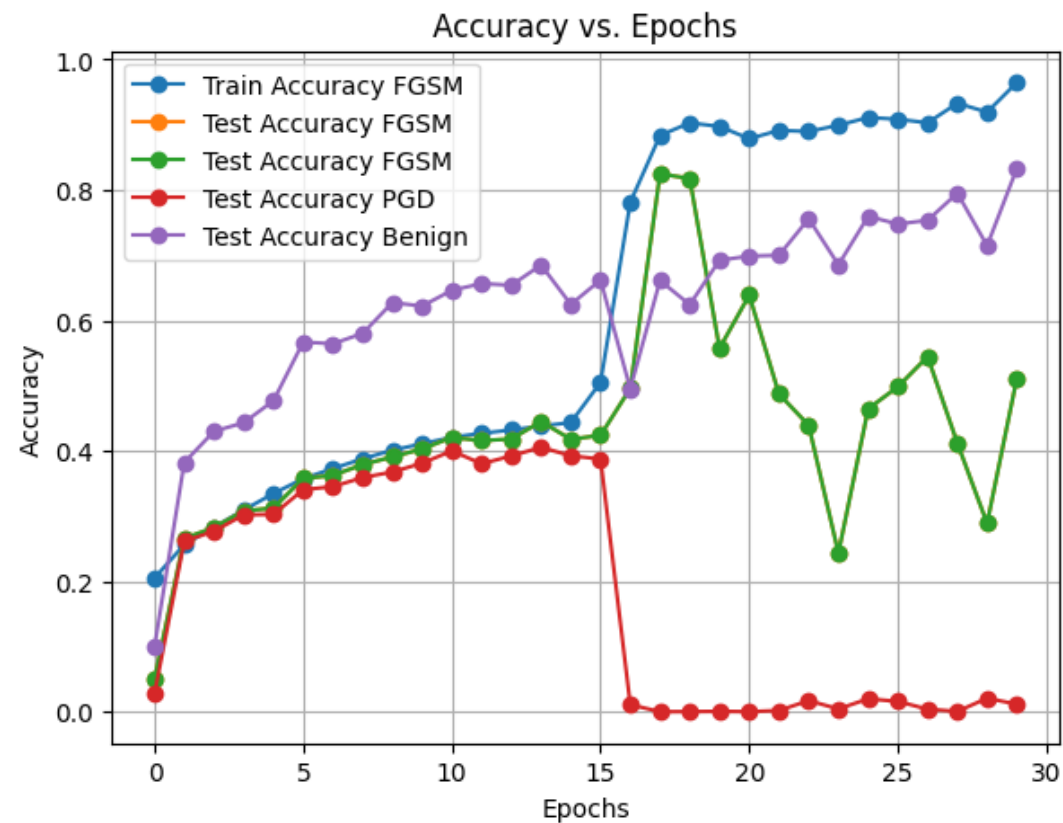
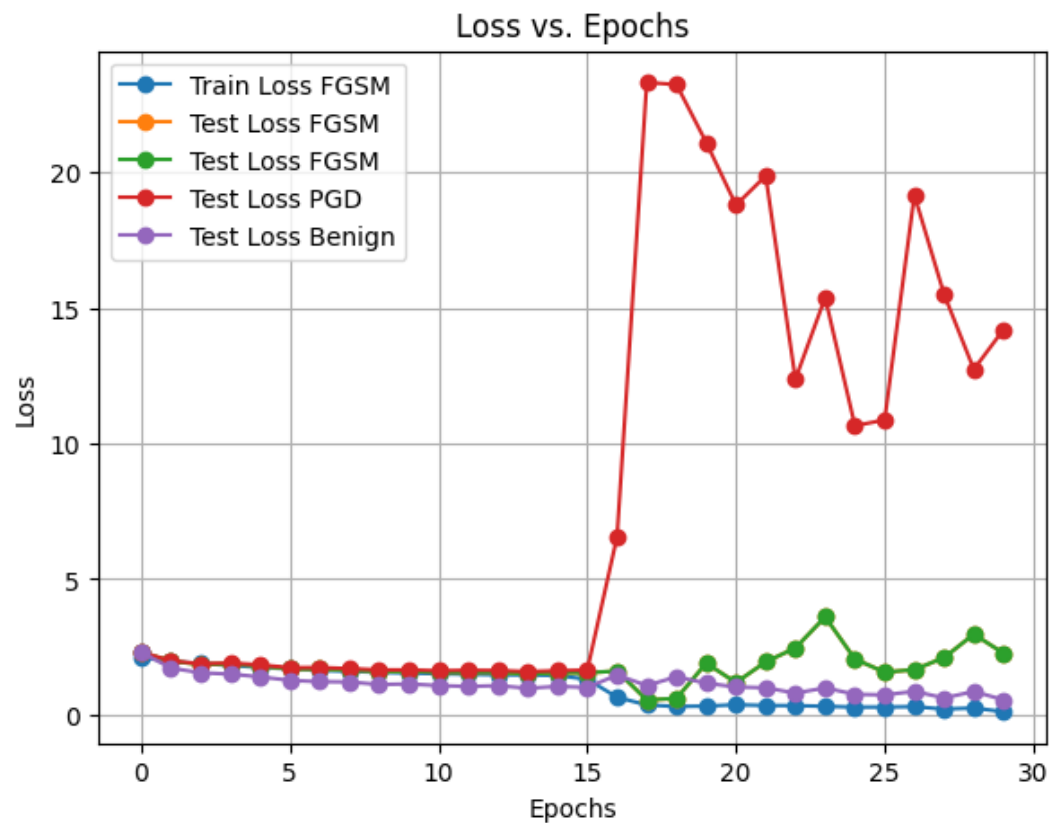
FGSM Alpha Drop (Epoch = 12)



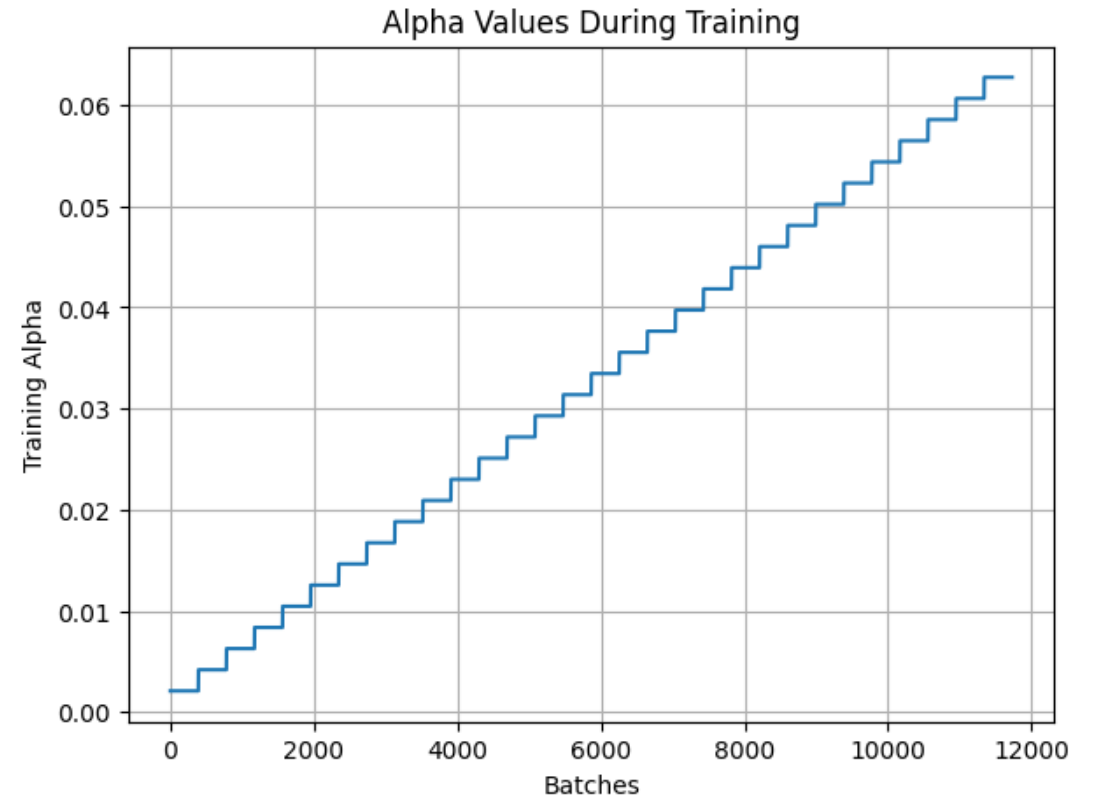
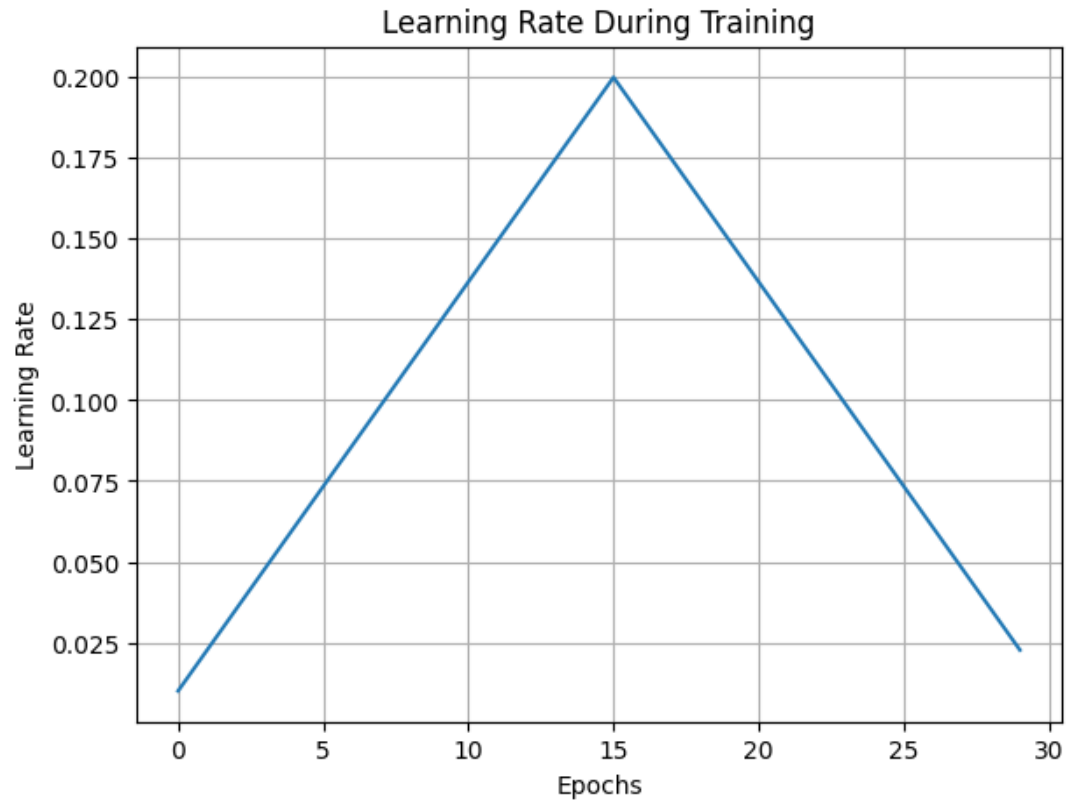
FGSM Alpha Drop (Epoch = 18)



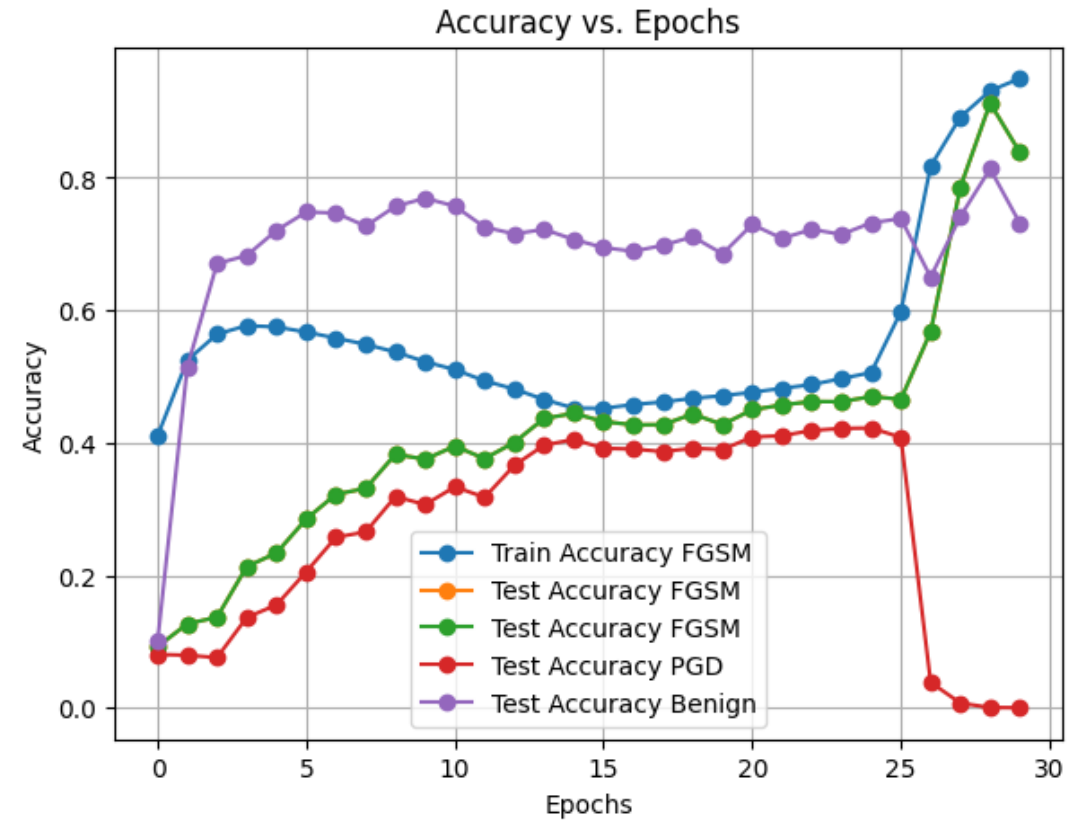
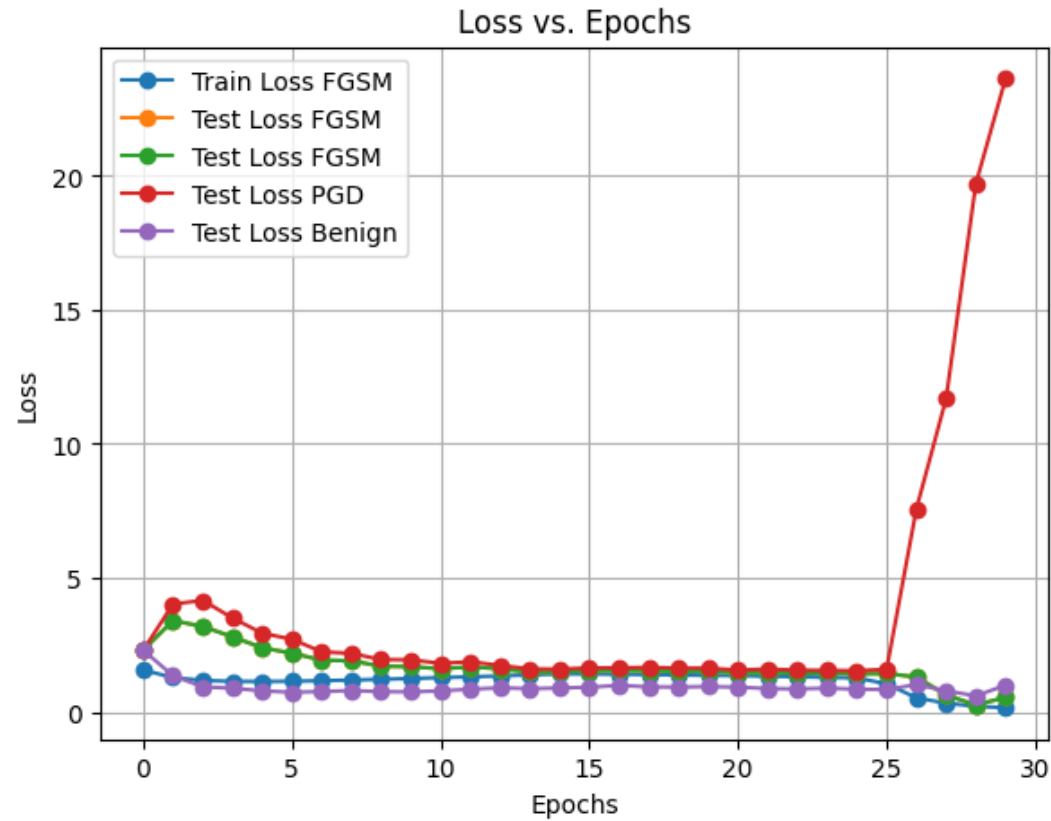
FGSM Alpha Drop (Epoch = 20)



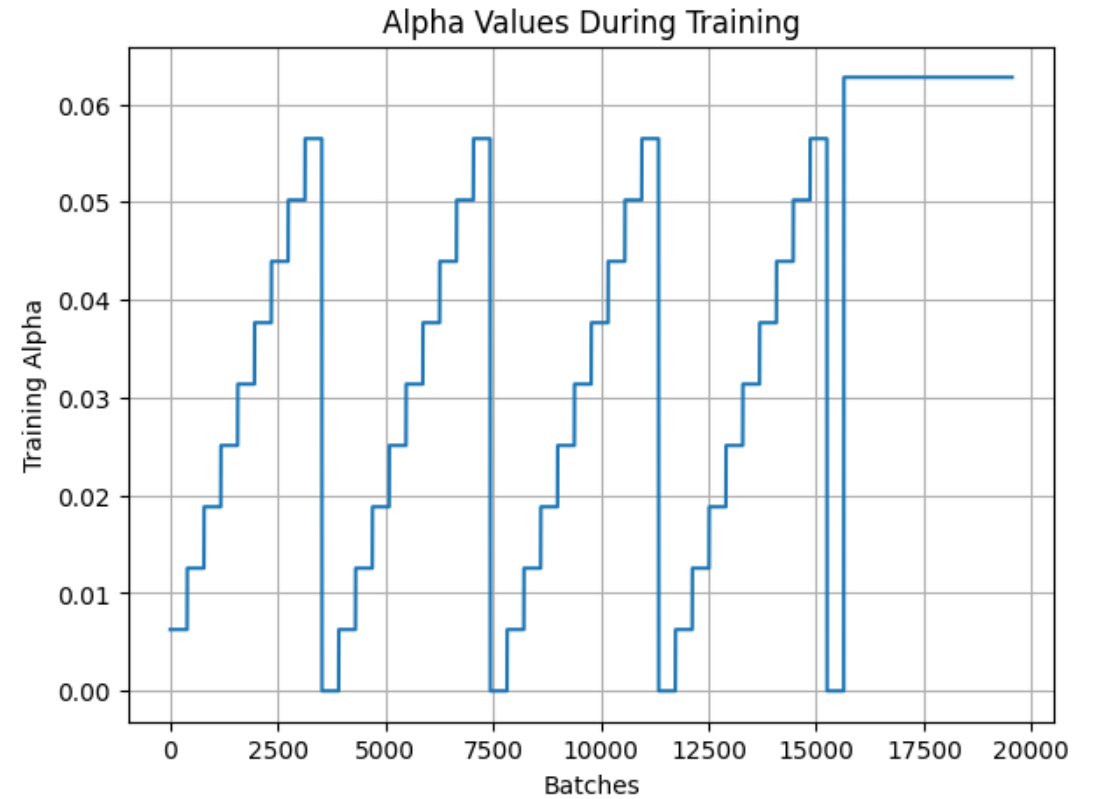
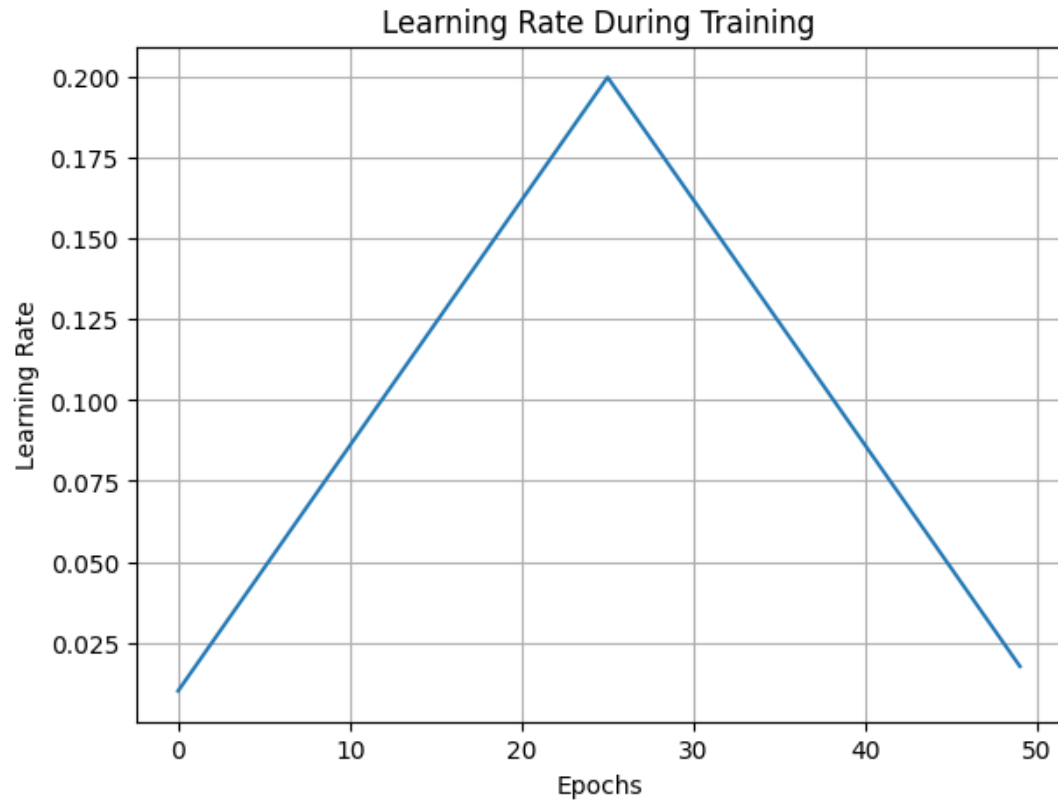
Scheduling FGSM Alpha During Training



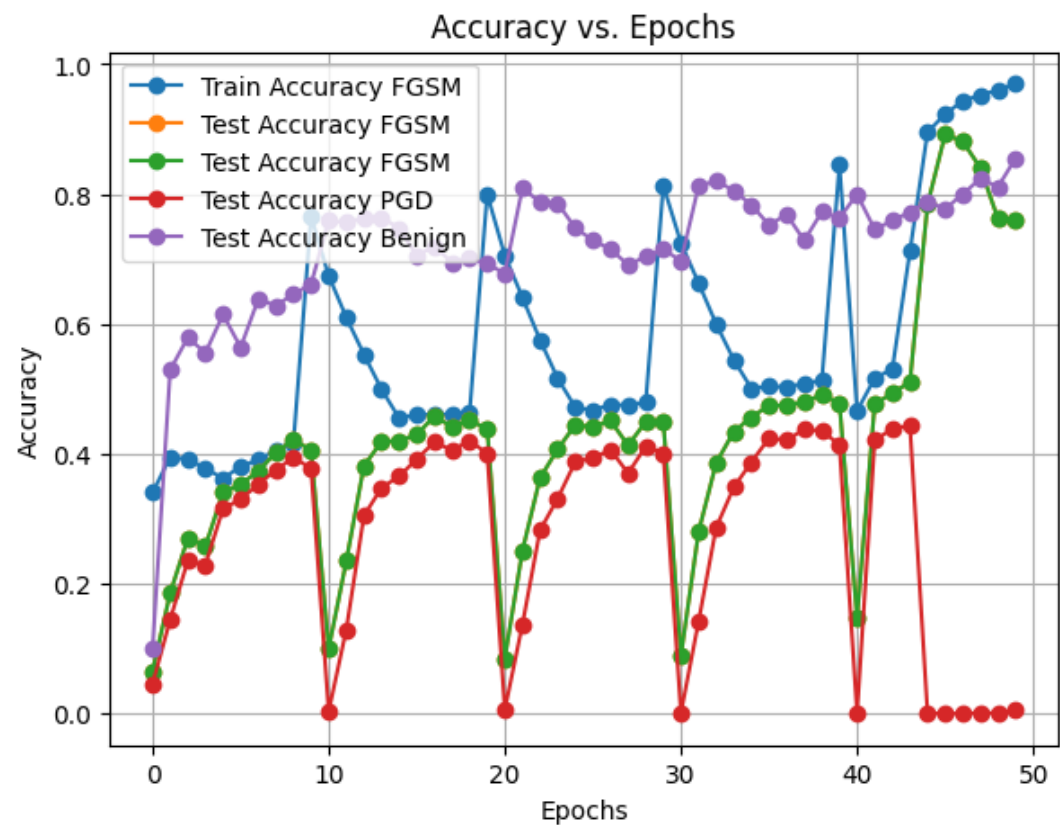
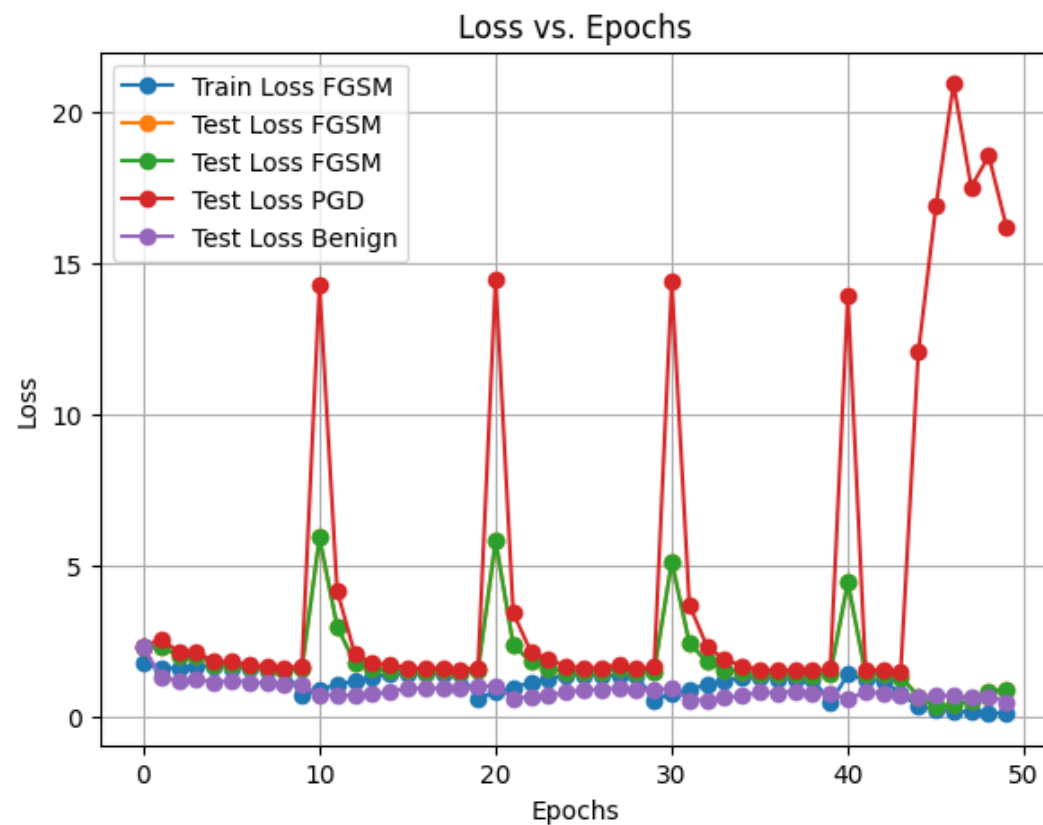
Scheduling FGSM Alpha During Training



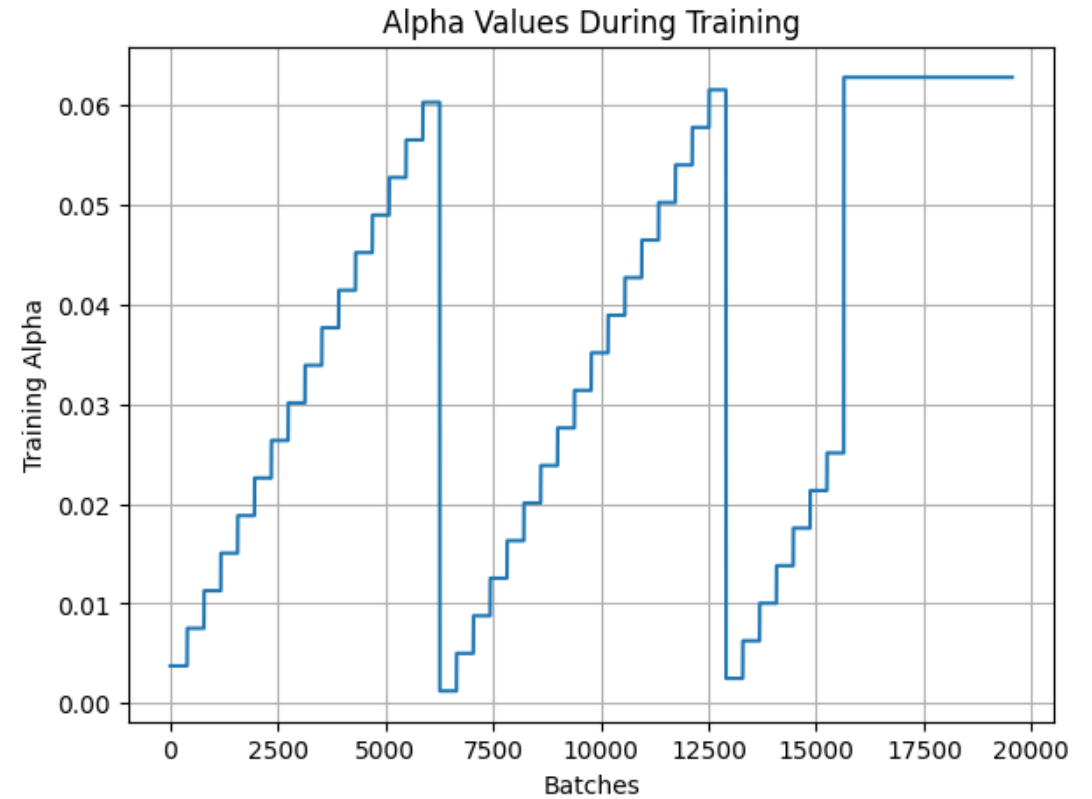
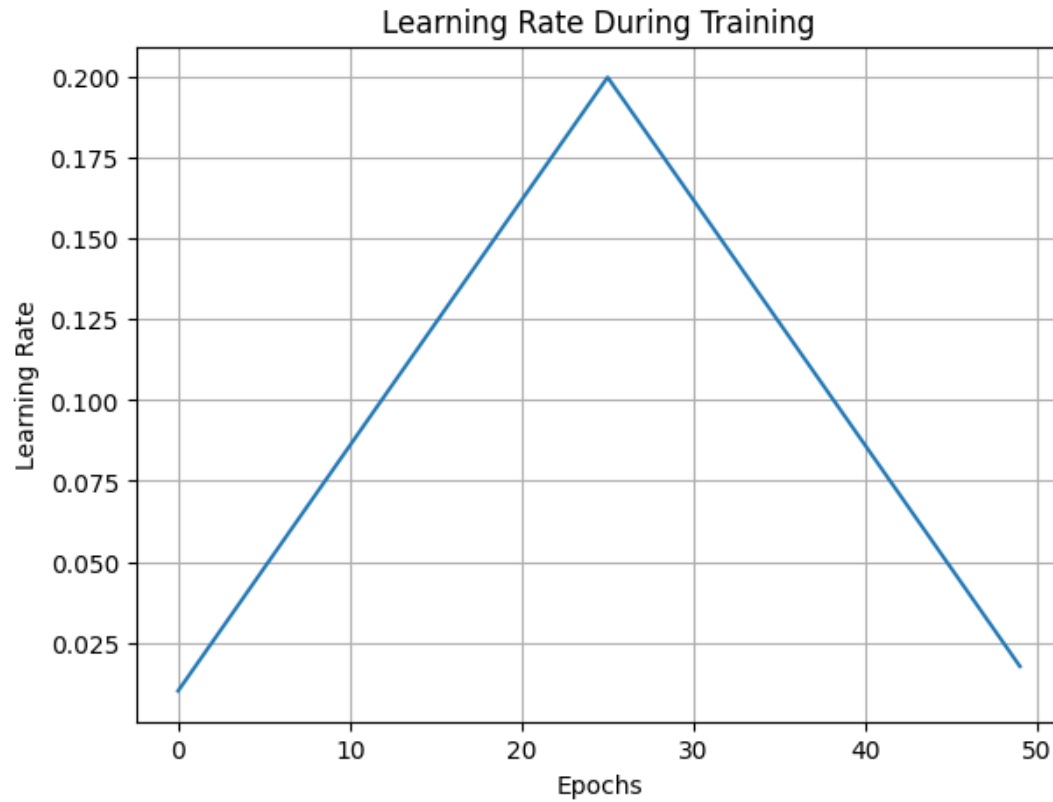
Scheduling FGSM Alpha During Training



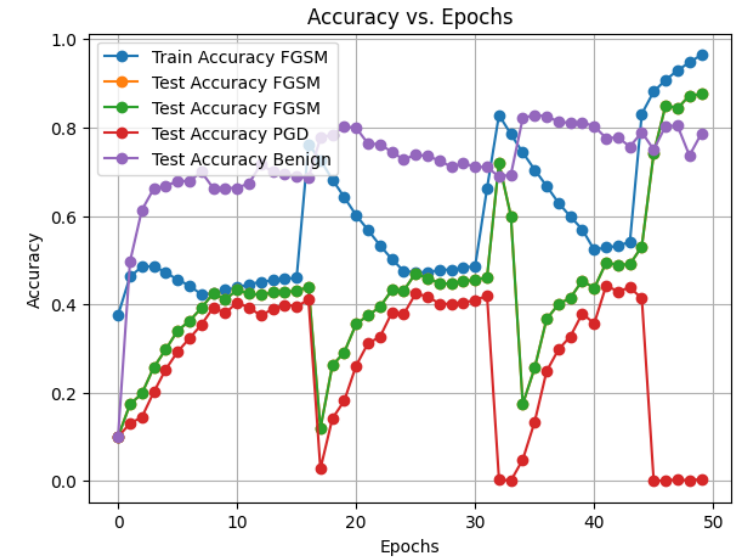
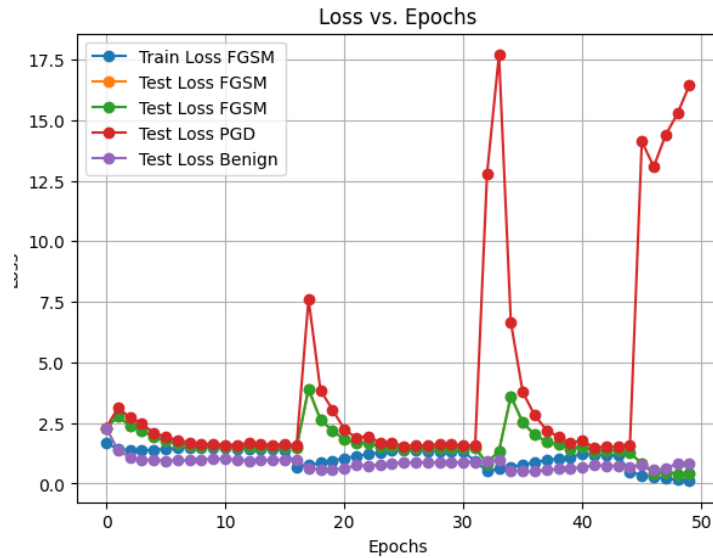
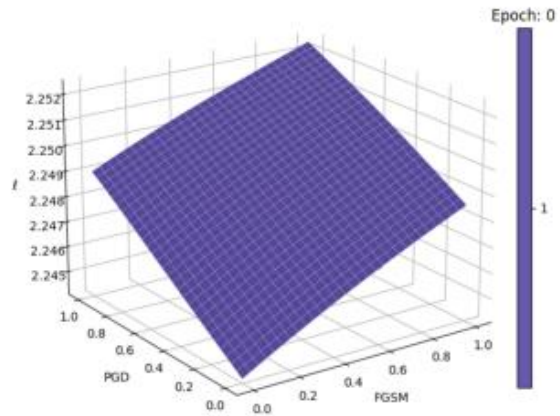
Scheduling FGSM Alpha During Training



Scheduling FGSM Alpha During Training



Scheduling FGSM Alpha During Training



Effect of BatchNorm on CO

